

BÁO CÁO THIẾT KẾ HỆ THỐNG SỐ VỚI HDL - LỚP CE213.Q23.2

Bài thực hành số 4

Giảng viên hướng dẫn	Hồ Ngọc Diễm		Điểm
Sinh viên thực hiện	Vũ Đại Dương	23520359	
	Nguyễn Đặng Phương Duy	23520372	

Câu 1: Thiết kế bộ ALU 32-bit với các chức năng sau:

M	S ₁	S ₀	ALU Operations
0	0	0	Complement A
0	0	1	AND
0	1	0	EX-OR
0	1	1	OR
1	0	0	Decrement A
1	0	1	Add
1	1	0	Subtract
1	1	1	Increment A

Hình 1: Chức năng của ALU

Hiện thực hóa thiết kế:

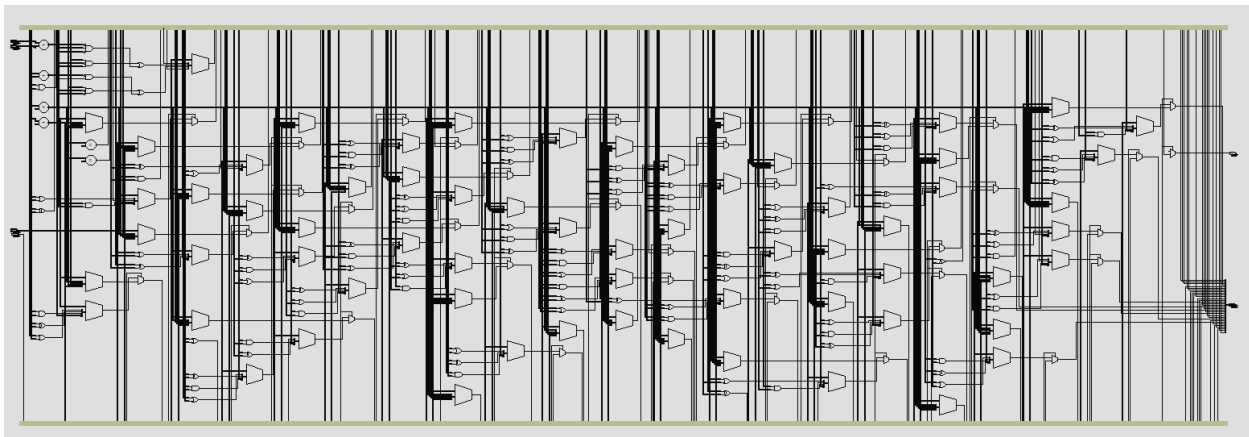
```
module bai1(  
    input wire [31:0] A, B,  
    input wire M,  
    input wire [1:0] S,  
    output reg [31:0] Y,  
    output reg add_sub_overflow  
);  
  
    localparam max_int = 32'h7FFFFFFF;  
    localparam min_int = 32'h80000000;  
  
    always@(*) begin  
        add_sub_overflow = 0;  
        if(M) begin  
            case(S)  
                2'b00: begin
```

```

        Y = A - 1;
        add_sub_overflow = (Y == max_int);
    end
    2'b01: begin
        Y = A + B;
        add_sub_overflow = (~A[31] & ~B[31] & Y[31]) |
(A[31] & B[31] & ~Y[31]);
    end
    2'b10: begin
        Y = A - B;
        add_sub_overflow = (~A[31] & B[31] & Y[31]) | (A[31]
& ~B[31] & ~Y[31]);
    end
    2'b11: begin
        Y = A + 1;
        add_sub_overflow = (Y == min_int);
    end
endcase
    end
    else begin
        case(S)
            2'b00: Y = ~A;
            2'b01: Y = A & B;
            2'b10: Y = A ^ B;
            2'b11: Y = A | B;
        endcase
        add_sub_overflow = 0;
    end
end
endmodule

```

Bảng 1 Code verilog ALU 32-bit



Hình 2: RTL Viewer của thiết kế

Mô phỏng chức năng và timing của thiết kế:

```
`timescale 1ns/1ps

module tb_bail;

    reg [31:0] A, B;
    reg M;
    reg [1:0] S;
    wire [31:0] Y;
    wire add_sub_overflow;

    // DUT
    bail uut (
        .A(A),
        .B(B),
        .M(M),
        .S(S),
        .Y(Y),
        .add_sub_overflow(add_sub_overflow)
    );

    // =====
    // MONITOR: theo dõi liên tục
    // =====
    initial begin
        $monitor("Time=%0t | M=%b S=%b | A=%h B=%h || Y=%h | OVF=%b",
            $time, M, S, A, B, Y, add_sub_overflow);
    end

    // =====
    // TASK: apply stimulus
    // =====
    task apply_test;
        input [31:0] a, b;
        input m;
        input [1:0] s;
    begin
        // gán input
        A = a;
        B = b;
        M = m;
        S = s;

        // in thông tin test case (display)
        $display("\n>>> APPLY: M=%b S=%b | A=%h B=%h",
            m, s, a, b);
    end
endmodule
```

```

// delay lớn để đảm bảo ổn định (GLS slow model)
#200;

// in kết quả cuối cùng (display)
$display(">>> RESULT: Y=%h | OVF=%b\n", Y, add_sub_overflow);
end
endtask

// =====
// MAIN TEST
// =====
initial begin
    // Khởi tạo
    A = 0; B = 0; M = 0; S = 0;
    #200;

    // =====
    // LOGIC MODE
    // =====
    $display("\n===== TEST LOGIC (M=0) =====");

    apply_test(32'hAAAAAAAA, 32'h55555555, 0, 2'b00); // ~A
    apply_test(32'hAAAAAAAA, 32'h55555555, 0, 2'b01); // A&B
    apply_test(32'hAAAAAAAA, 32'h55555555, 0, 2'b10); // A^B
    apply_test(32'hAAAAAAAA, 32'h55555555, 0, 2'b11); // A|B

    // =====
    // ARITHMETIC MODE
    // =====
    $display("\n===== TEST ARITHMETIC (M=1) =====");

    apply_test(32'd100, 32'd50, 1, 2'b00); // A-1
    apply_test(32'd100, 32'd50, 1, 2'b01); // A+B
    apply_test(32'd100, 32'd50, 1, 2'b10); // A-B
    apply_test(32'd100, 32'd50, 1, 2'b11); // A+1

    // =====
    // OVERFLOW TEST
    // =====
    $display("\n===== TEST OVERFLOW =====");

    apply_test(32'h7FFFFFFF, 32'h2, 1, 2'b01); // max + pos
    apply_test(32'h80000000, 32'h2, 1, 2'b10); // min - pos
    apply_test(32'h7FFFFFFF, 32'h2, 1, 2'b11); // max + 1
    apply_test(32'h80000000, 32'h2, 1, 2'b00); // min - 1

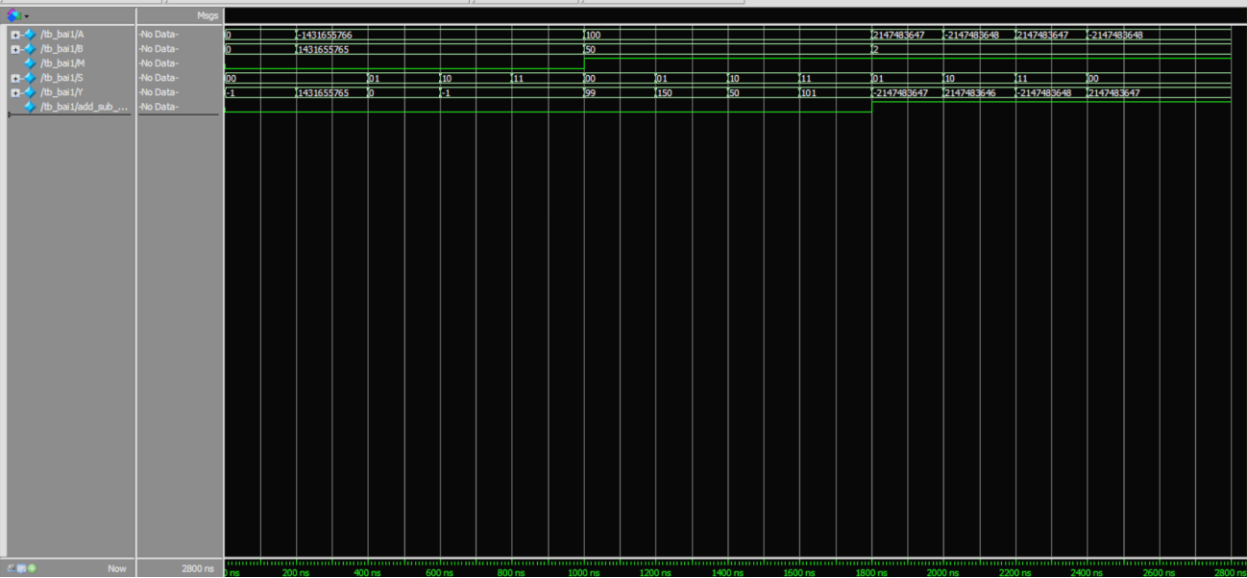
```

```
$display("\n===== END SIMULATION =====");
#200;
$finish;
end

endmodule
```

Bảng 2 Code testbench mô phỏng thiết kế

Kết quả khi chạy mô phỏng functional:



Hình 3 Waveform mô phỏng functional

```
Time=0 | M=0 S=00 | A=00000000 B=00000000 || Y=ffffff | OVF=0
#
# ===== TEST LOGIC (M=0) =====
#
# >>> APPLY: M=0 S=00 | A=aaaaaaaa B=55555555
# Time=200000 | M=0 S=00 | A=aaaaaaaa B=55555555 || Y=55555555 | OVF=0
# >>> RESULT: Y=55555555 | OVF=0
#
#
# >>> APPLY: M=0 S=01 | A=aaaaaaaa B=55555555
# Time=400000 | M=0 S=01 | A=aaaaaaaa B=55555555 || Y=00000000 | OVF=0
# >>> RESULT: Y=00000000 | OVF=0
#
#
# >>> APPLY: M=0 S=10 | A=aaaaaaaa B=55555555
# Time=600000 | M=0 S=10 | A=aaaaaaaa B=55555555 || Y=ffffff | OVF=0
# >>> RESULT: Y=ffffff | OVF=0
#
#
# >>> APPLY: M=0 S=11 | A=aaaaaaaa B=55555555
# Time=800000 | M=0 S=11 | A=aaaaaaaa B=55555555 || Y=ffffff | OVF=0
# >>> RESULT: Y=ffffff | OVF=0
#
#
# ===== TEST ARITHMETIC (M=1) =====
#
# >>> APPLY: M=1 S=00 | A=00000064 B=00000032
# Time=1000000 | M=1 S=00 | A=00000064 B=00000032 || Y=00000063 | OVF=0
# >>> RESULT: Y=00000063 | OVF=0
#
#
# >>> APPLY: M=1 S=01 | A=00000064 B=00000032
# Time=1200000 | M=1 S=01 | A=00000064 B=00000032 || Y=00000096 | OVF=0
# >>> RESULT: Y=00000096 | OVF=0
#
#
# >>> APPLY: M=1 S=10 | A=00000064 B=00000032
# Time=1400000 | M=1 S=10 | A=00000064 B=00000032 || Y=00000032 | OVF=0
# >>> RESULT: Y=00000032 | OVF=0
#
#
# >>> APPLY: M=1 S=11 | A=00000064 B=00000032
# Time=1600000 | M=1 S=11 | A=00000064 B=00000032 || Y=00000065 | OVF=0
# >>> RESULT: Y=00000065 | OVF=0
#
#
```

```

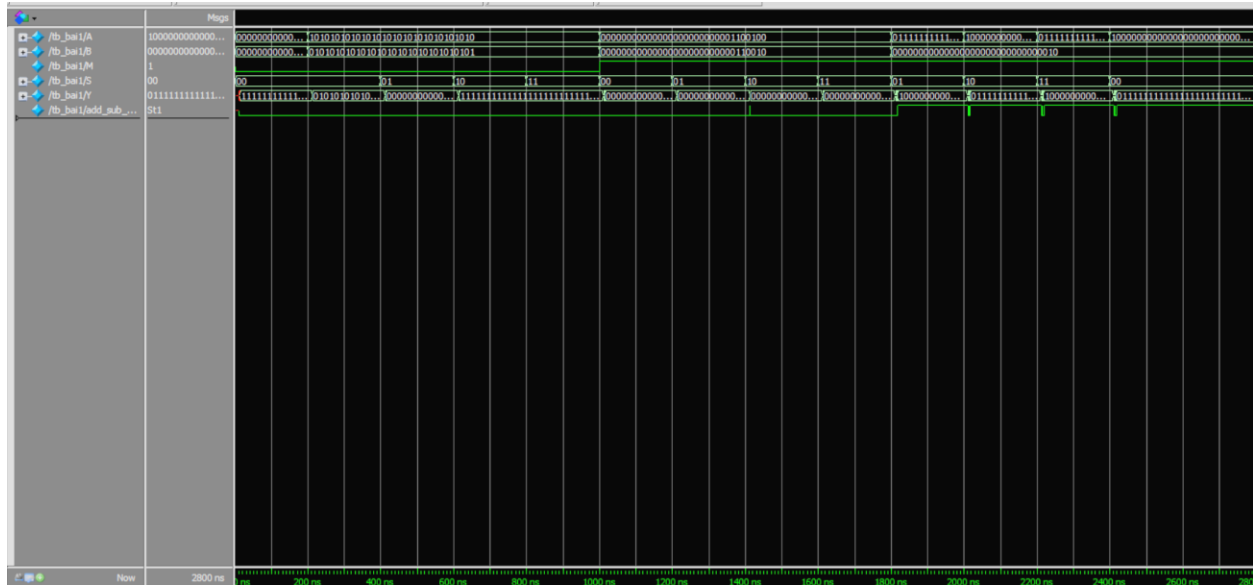
# ===== TEST OVERFLOW =====
#
# >>> APPLY: M=1 S=01 | A=7ffffff B=00000002
# Time=1800000 | M=1 S=01 | A=7ffffff B=00000002 || Y=80000001 | OVF=1
# >>> RESULT: Y=80000001 | OVF=1
#
#
# >>> APPLY: M=1 S=10 | A=80000000 B=00000002
# Time=2000000 | M=1 S=10 | A=80000000 B=00000002 || Y=7ffffffe | OVF=1
# >>> RESULT: Y=7ffffffe | OVF=1
#
#
# >>> APPLY: M=1 S=11 | A=7ffffff B=00000002
# Time=2200000 | M=1 S=11 | A=7ffffff B=00000002 || Y=80000000 | OVF=1
# >>> RESULT: Y=80000000 | OVF=1
#
#
# >>> APPLY: M=1 S=00 | A=80000000 B=00000002
# Time=2400000 | M=1 S=00 | A=80000000 B=00000002 || Y=7ffffff | OVF=1
# >>> RESULT: Y=7ffffff | OVF=1
#
#
# ===== END SIMULATION =====

```

Nhận xét:

- Nhìn vào Waveform, có thể thấy các tín hiệu hiển thị cách nhau mỗi 20ns → theo đúng code và kết quả đúng
 - Nhìn vào Console, có thể thấy đã hiển thị các dòng tại lệnh \$display và \$monitor, đồng thời hiển thị kết quả trực quan hơn
 - Kết quả tính toán đã đúng theo bảng chức năng ALU
- ⇒ Mô phỏng chức năng đã đáp ứng yêu cầu

Kết quả khi chạy mô phỏng timing:



Hình 4: Waveform mô phỏng timing

```
# Time=0 | M=0 S=00 | A=00000000 B=00000000 || Y=xxxxxxx | OVF=x
...
# Time=12000 | M=0 S=00 | A=00000000 B=00000000 || Y=ffffff | OVF=0
#
# ===== TEST LOGIC (M=0) =====
#
# >>> APPLY: M=0 S=00 | A=aaaaaaa B=55555555
# Time=200000 | M=0 S=00 | A=aaaaaaa B=55555555 || Y=ffffff | OVF=0
...
# Time=213000 | M=0 S=00 | A=aaaaaaa B=55555555 || Y=555d5555 | OVF=0
# Time=213000 | M=0 S=00 | A=aaaaaaa B=55555555 || Y=55555555 | OVF=0
# >>> RESULT: Y=55555555 | OVF=0
#
#
# >>> APPLY: M=0 S=01 | A=aaaaaaa B=55555555
# Time=400000 | M=0 S=01 | A=aaaaaaa B=55555555 || Y=55555555 | OVF=0
...
# Time=412000 | M=0 S=01 | A=aaaaaaa B=55555555 || Y=00404000 | OVF=0
# Time=412000 | M=0 S=01 | A=aaaaaaa B=55555555 || Y=00004000 | OVF=0
# Time=412000 | M=0 S=01 | A=aaaaaaa B=55555555 || Y=00000000 | OVF=0
# >>> RESULT: Y=00000000 | OVF=0
#
#
# >>> APPLY: M=0 S=10 | A=aaaaaaa B=55555555
# Time=600000 | M=0 S=10 | A=aaaaaaa B=55555555 || Y=00000000 | OVF=0
# Time=612000 | M=0 S=10 | A=aaaaaaa B=55555555 || Y=00001000 | OVF=0
...
# Time=613000 | M=0 S=10 | A=aaaaaaa B=55555555 || Y=fff7ffff | OVF=0
```



```

# Time=613000 | M=0 S=10 | A=aaaaaaaa B=55555555 || Y=ffffff | OVF=0
# >>> RESULT: Y=ffffff | OVF=0
#
#
# >>> APPLY: M=0 S=11 | A=aaaaaaaa B=55555555
# Time=800000 | M=0 S=11 | A=aaaaaaaa B=55555555 || Y=ffffff | OVF=0
# >>> RESULT: Y=ffffff | OVF=0
#
#
# ===== TEST ARITHMETIC (M=1) =====
#
# >>> APPLY: M=1 S=00 | A=00000064 B=00000032
# Time=1000000 | M=1 S=00 | A=00000064 B=00000032 || Y=ffffff | OVF=0
# Time=1009000 | M=1 S=00 | A=00000064 B=00000032 || Y=ffffffe | OVF=0
...
# Time=1015000 | M=1 S=00 | A=00000064 B=00000032 || Y=00008063 | OVF=0
# Time=1015000 | M=1 S=00 | A=00000064 B=00000032 || Y=00000063 | OVF=0
# >>> RESULT: Y=00000063 | OVF=0
#
#
# >>> APPLY: M=1 S=01 | A=00000064 B=00000032
# Time=1200000 | M=1 S=01 | A=00000064 B=00000032 || Y=00000063 | OVF=0
...
# Time=1214000 | M=1 S=01 | A=00000064 B=00000032 || Y=00000096 | OVF=0
# >>> RESULT: Y=00000096 | OVF=0
#
#
# >>> APPLY: M=1 S=10 | A=00000064 B=00000032
# Time=1400000 | M=1 S=10 | A=00000064 B=00000032 || Y=00000096 | OVF=0
# Time=1412000 | M=1 S=10 | A=00000064 B=00000032 || Y=00000096 | OVF=1
...
# Time=1413000 | M=1 S=10 | A=00000064 B=00000032 || Y=000000b2 | OVF=0
# Time=1413000 | M=1 S=10 | A=00000064 B=00000032 || Y=00000032 | OVF=0
# >>> RESULT: Y=00000032 | OVF=0
#
#
# >>> APPLY: M=1 S=11 | A=00000064 B=00000032
# Time=1600000 | M=1 S=11 | A=00000064 B=00000032 || Y=00000032 | OVF=0
# Time=1612000 | M=1 S=11 | A=00000064 B=00000032 || Y=00000033 | OVF=0
# Time=1613000 | M=1 S=11 | A=00000064 B=00000032 || Y=00000073 | OVF=0
# Time=1613000 | M=1 S=11 | A=00000064 B=00000032 || Y=00000071 | OVF=0
# Time=1613000 | M=1 S=11 | A=00000064 B=00000032 || Y=00000075 | OVF=0
# Time=1613000 | M=1 S=11 | A=00000064 B=00000032 || Y=00000065 | OVF=0
# >>> RESULT: Y=00000065 | OVF=0
#
#

```

```

# ===== TEST OVERFLOW =====
#
# >>> APPLY: M=1 S=01 | A=7ffffff B=00000002
# Time=1800000 | M=1 S=01 | A=7ffffff B=00000002 || Y=00000065 | OVF=0
# Time=1809000 | M=1 S=01 | A=7ffffff B=00000002 || Y=00000064 | OVF=0
...
# Time=1816000 | M=1 S=01 | A=7ffffff B=00000002 || Y=98000001 | OVF=0
# Time=1817000 | M=1 S=01 | A=7ffffff B=00000002 || Y=98000001 | OVF=1
# Time=1817000 | M=1 S=01 | A=7ffffff B=00000002 || Y=90000001 | OVF=1
# Time=1817000 | M=1 S=01 | A=7ffffff B=00000002 || Y=80000001 | OVF=1
# >>> RESULT: Y=80000001 | OVF=1
#
#
# >>> APPLY: M=1 S=10 | A=80000000 B=00000002
# Time=2000000 | M=1 S=10 | A=80000000 B=00000002 || Y=80000001 | OVF=1
# Time=2009000 | M=1 S=10 | A=80000000 B=00000002 || Y=80000000 | OVF=1
...
# Time=2017000 | M=1 S=10 | A=80000000 B=00000002 || Y=7ffffffe | OVF=0
# Time=2017000 | M=1 S=10 | A=80000000 B=00000002 || Y=7ffffffe | OVF=1
# >>> RESULT: Y=7ffffffe | OVF=1
#
#
# >>> APPLY: M=1 S=11 | A=7ffffff B=00000002
# Time=2200000 | M=1 S=11 | A=7ffffff B=00000002 || Y=7ffffffe | OVF=1
# Time=2209000 | M=1 S=11 | A=7ffffff B=00000002 || Y=7ffffff | OVF=1
...
# Time=2216000 | M=1 S=11 | A=7ffffff B=00000002 || Y=80000000 | OVF=0
# Time=2218000 | M=1 S=11 | A=7ffffff B=00000002 || Y=80000000 | OVF=1
# >>> RESULT: Y=80000000 | OVF=1
#
#
# >>> APPLY: M=1 S=00 | A=80000000 B=00000002
# Time=2400000 | M=1 S=00 | A=80000000 B=00000002 || Y=80000000 | OVF=1
# Time=2409000 | M=1 S=00 | A=80000000 B=00000002 || Y=80000001 | OVF=1
...
# Time=2417000 | M=1 S=00 | A=80000000 B=00000002 || Y=77fffff | OVF=0
# Time=2417000 | M=1 S=00 | A=80000000 B=00000002 || Y=7ffffff | OVF=0
# Time=2417000 | M=1 S=00 | A=80000000 B=00000002 || Y=7ffffff | OVF=1
# >>> RESULT: Y=7ffffff | OVF=1
#
#
# ===== END SIMULATION =====

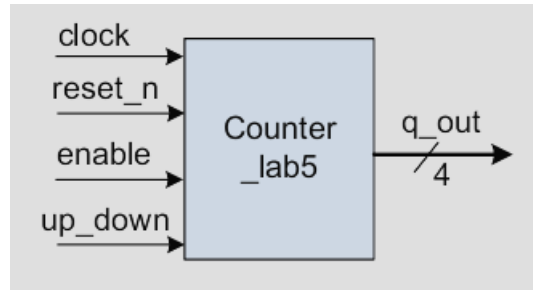
```

Nhận xét:

- Từ waveform ta có thể thấy tại các thời điểm S mà M thay đổi thì Y không thay đổi lập tức mà có các nhiễu chuyển trạng thái (output chưa phải là output cuối cùng), phải sau một lúc thì mới có output cuối cùng

- Từ kết quả console có thể thấy output được thay đổi rất nhiều và hiển thị thông qua monitor, chứng minh cho việc có delay khi S và M thay đổi

Câu 2: Thiết kế mạch đếm 4-bit có sơ đồ như bên dưới:



Hình 5: Mạch đếm 4 bit

Hiện thực hóa thiết kế:

```

module bai2(
    input wire clk, rst_n, en, up_down,
    output [3:0] q_out
);

    reg [3:0] cnt = 4'b0;
    assign q_out = cnt;
    always @(posedge clk or negedge rst_n) begin
        if(!rst_n) begin
            cnt <= 4'b0;
        end
        else begin
            if(en) begin
                if(up_down) begin
                    cnt <= cnt - 4'b1;
                end
                else begin
                    cnt <= cnt + 4'b1;
                end
            end
        end
    end
endmodule
  
```

Yêu cầu:

2.1 Viết testbench để kiểm tra thiết kế theo mô hình quan sát dạng sóng bằng phần mềm ModelSim-Altera

```
`timescale 1ns/1ps
```

```

module tb_bai2;

    reg clk;
    reg rst_n;
    reg en;
    reg up_down;

    wire [3:0] q_out;

    // DUT
    bai2 uut (
        .clk(clk),
        .rst_n(rst_n),
        .en(en),
        .up_down(up_down),
        .q_out(q_out)
    );

    // =====
    // CLOCK GENERATION
    // Chu kỳ clock = 10ns
    // =====
    initial begin
        clk = 0;
        forever #5 clk = ~clk;
    end

    // =====
    // MONITOR
    // =====
    initial begin
        $monitor("Time=%0t | rst_n=%b en=%b up_down=%b | q_out=%b (%0d)",
            $time, rst_n, en, up_down, q_out, q_out);
    end

    // =====
    // TEST STIMULUS
    // =====
    initial begin

        // Khởi tạo
        rst_n = 0;
        en = 0;
        up_down = 0;
    end

```

```

// Reset
#12;
rst_n = 1;

// =====
// TEST COUNT UP
// =====
$display("\n===== TEST COUNT UP =====");

en = 1;
up_down = 0;

#200;

// =====
// TEST COUNT DOWN
// =====
$display("\n===== TEST COUNT DOWN =====");

up_down = 1;

#200;

// =====
// TEST ENABLE = 0
// =====
$display("\n===== TEST HOLD =====");

en = 0;

#50;

// =====
// RESET AGAIN
// =====
$display("\n===== TEST RESET =====");

rst_n = 1;
en = 1;
up_down = 0;

#50;

rst_n = 0;

#10;

```

```

        rst_n = 1;
    en = 1;
    up_down = 0;

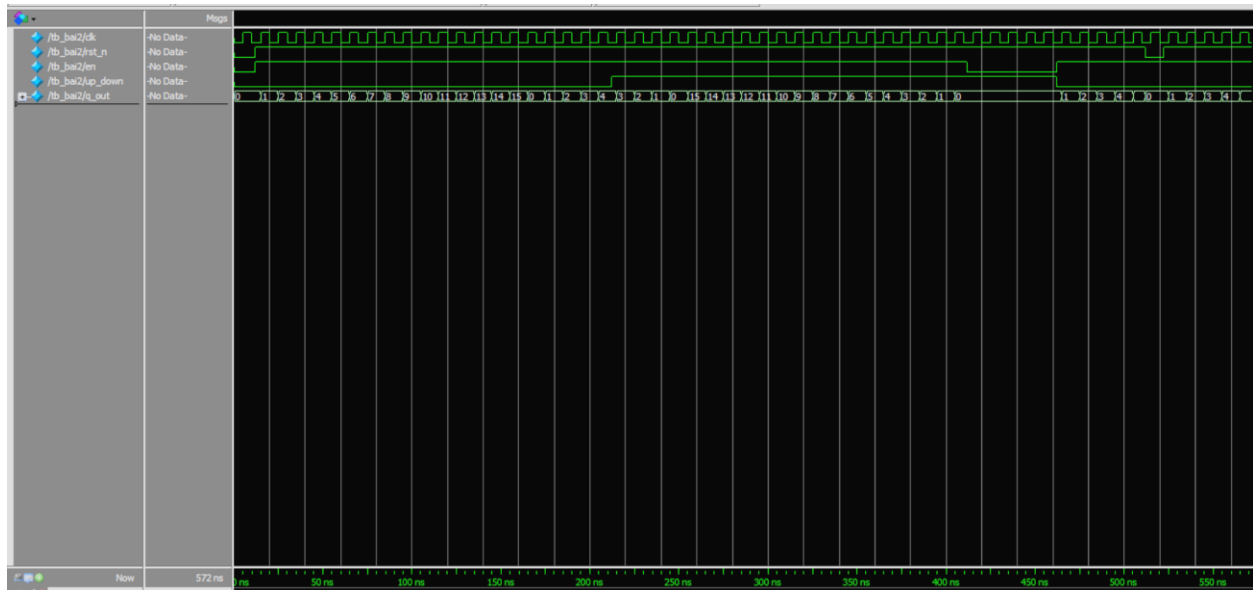
    #50;

    $display("\n===== END SIMULATION =====");

    $stop;
end
endmodule

```

Kết quả mô phỏng functional:



Hình 6: Waveform kết quả mạch đếm functional

Nhận xét: Từ wave form có thể thấy mạch đã hoạt động đúng chức năng của mạch đếm

2.2 Viết testbench để kiểm tra thiết kế theo mô hình tự kiểm tra (Self-checking) bằng phần mềm ModelSim-Altera

```

`timescale 1ns/1ps

module tb_bai2_sc;

    // --- 1. Khai báo tín hiệu kết nối với DUT ---
    reg clk;
    reg rst_n;

```

```

reg en;
reg up_down;
wire [3:0] q_out;

// --- 2. Biến phục vụ Self-checking (Golden Model) ---
reg [3:0] expected_q;
integer error_count = 0;

// --- 3. Khởi tạo DUT (Device Under Test) ---
bai2 uut (
    .clk(clk),
    .rst_n(rst_n),
    .en(en),
    .up_down(up_down),
    .q_out(q_out)
);

// --- 4. Tạo xung Clock (Chu kỳ 10ns - 100MHz) ---
initial begin
    clk = 0;
    forever #5 clk = ~clk;
end

// --- 5. MONITOR: Hiển thị thay đổi ra màn hình ---
initial begin
    $monitor("Time=%0t | rst_n=%b en=%b up_down=%b | q_out=%d | expected=%d",
        $time, rst_n, en, up_down, q_out, expected_q);
end

// --- 6. GOLDEN MODEL: Tính toán giá trị mong đợi ---
always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        expected_q <= 4'b0;
    end
    else if (en) begin
        if (up_down)
            expected_q <= expected_q - 1'b1;
        else
            expected_q <= expected_q + 1'b1;
    end
end

// --- 7. SELF-CHECKING LOGIC ---
// Kiểm tra tại mốc 8ns sau mỗi cạnh lên (khi chu kỳ là 10ns)
// Việc delay này giúp tránh các lỗi do độ trễ cổng (Gate Delay) và nhiễu (Glitch)
always @(posedge clk) begin

```

```

#8;
if (rst_n === 1'b1) begin // Chỉ kiểm tra khi đã thoát khỏi trạng thái Reset
    if (q_out !== expected_q) begin
        $display(">>> ERROR at %0t: Mismatch! Real=%d, Expected=%d", $time, q_out,
expected_q);
        error_count = error_count + 1;
    end
end
end
end

// --- 8. TEST STIMULUS (Kích bản kiểm tra) ---
initial begin
    // Khởi tạo ban đầu
    rst_n = 0;
    en = 0;
    up_down = 0;

    // Reset hệ thống trong 1.2 chu kỳ để đảm bảo mọi thứ về 0
    #12 rst_n = 1;

    // CASE 1: Đếm lên (UP)
    $display("\n===== TEST CASE 1: COUNT UP =====");
    en = 1;
    up_down = 0;
    #100; // Chạy trong 10 chu kỳ

    // CASE 2: Đếm xuống (DOWN)
    $display("\n===== TEST CASE 2: COUNT DOWN =====");
    up_down = 1;
    #100;

    // CASE 3: Giữ giá trị (ENABLE = 0)
    $display("\n===== TEST CASE 3: HOLD (EN=0) =====");
    en = 0;
    #50;

    // CASE 4: Reset bất đồng bộ khi đang chạy
    $display("\n===== TEST CASE 4: ASYNC RESET =====");
    en = 1;
    up_down = 0;
    #30;
    rst_n = 0; // Kích hoạt Reset
    #10;
    rst_n = 1; // Nhả Reset
    #50;

```



```

// --- TỔNG KẾT ---
$display("\n=====");
$display(" SIMULATION FINISHED");
if (error_count == 0)
    $display(" RESULT: PASSED! No errors found.");
else
    $display(" RESULT: FAILED! Total errors: %d", error_count);
$display("=====");

$stop;
end

endmodule

```

Kết quả chạy mô phỏng functional:

```

# Time=0 | rst_n=0 en=0 up_down=0 | q_out= 0 | expected= 0
#
# ===== TEST CASE 1: COUNT UP =====
# Time=12000 | rst_n=1 en=1 up_down=0 | q_out= 0 | expected= 0
# Time=15000 | rst_n=1 en=1 up_down=0 | q_out= 1 | expected= 1
# Time=25000 | rst_n=1 en=1 up_down=0 | q_out= 2 | expected= 2
# Time=35000 | rst_n=1 en=1 up_down=0 | q_out= 3 | expected= 3
# Time=45000 | rst_n=1 en=1 up_down=0 | q_out= 4 | expected= 4
# Time=55000 | rst_n=1 en=1 up_down=0 | q_out= 5 | expected= 5
# Time=65000 | rst_n=1 en=1 up_down=0 | q_out= 6 | expected= 6
# Time=75000 | rst_n=1 en=1 up_down=0 | q_out= 7 | expected= 7
# Time=85000 | rst_n=1 en=1 up_down=0 | q_out= 8 | expected= 8
# Time=95000 | rst_n=1 en=1 up_down=0 | q_out= 9 | expected= 9
# Time=105000 | rst_n=1 en=1 up_down=0 | q_out=10 | expected=10
#
# ===== TEST CASE 2: COUNT DOWN =====
# Time=112000 | rst_n=1 en=1 up_down=1 | q_out=10 | expected=10
# Time=115000 | rst_n=1 en=1 up_down=1 | q_out= 9 | expected= 9
# Time=125000 | rst_n=1 en=1 up_down=1 | q_out= 8 | expected= 8
# Time=135000 | rst_n=1 en=1 up_down=1 | q_out= 7 | expected= 7
# Time=145000 | rst_n=1 en=1 up_down=1 | q_out= 6 | expected= 6
# Time=155000 | rst_n=1 en=1 up_down=1 | q_out= 5 | expected= 5
# Time=165000 | rst_n=1 en=1 up_down=1 | q_out= 4 | expected= 4
# Time=175000 | rst_n=1 en=1 up_down=1 | q_out= 3 | expected= 3
# Time=185000 | rst_n=1 en=1 up_down=1 | q_out= 2 | expected= 2
# Time=195000 | rst_n=1 en=1 up_down=1 | q_out= 1 | expected= 1
# Time=205000 | rst_n=1 en=1 up_down=1 | q_out= 0 | expected= 0
#
# ===== TEST CASE 3: HOLD (EN=0) =====
# Time=212000 | rst_n=1 en=0 up_down=1 | q_out= 0 | expected= 0

```

```
#
# ===== TEST CASE 4: ASYNC RESET =====
# Time=262000 | rst_n=1 en=1 up_down=0 | q_out= 0 | expected= 0
# Time=265000 | rst_n=1 en=1 up_down=0 | q_out= 1 | expected= 1
# Time=275000 | rst_n=1 en=1 up_down=0 | q_out= 2 | expected= 2
# Time=285000 | rst_n=1 en=1 up_down=0 | q_out= 3 | expected= 3
# Time=292000 | rst_n=0 en=1 up_down=0 | q_out= 0 | expected= 0
# Time=302000 | rst_n=1 en=1 up_down=0 | q_out= 0 | expected= 0
# Time=305000 | rst_n=1 en=1 up_down=0 | q_out= 1 | expected= 1
# Time=315000 | rst_n=1 en=1 up_down=0 | q_out= 2 | expected= 2
# Time=325000 | rst_n=1 en=1 up_down=0 | q_out= 3 | expected= 3
# Time=335000 | rst_n=1 en=1 up_down=0 | q_out= 4 | expected= 4
# Time=345000 | rst_n=1 en=1 up_down=0 | q_out= 5 | expected= 5
#
# =====
# SIMULATION FINISHED
# RESULT: PASSED! No errors found.
# =====
```

Nhận xét: Test bench đã chạy đúng và kiểm tra đúng hoạt động của mạch

Câu 3: Sử dụng ngôn ngữ Verilog HDL, thiết kế một tập gồm 32 thanh ghi, mỗi thanh ghi 4 byte. Tập thanh ghi có 2 port đọc, 1 port ghi, gồm các tín hiệu sau: ReadAddress1[4:0], ReadAddress2[4:0], WriteAddress[4:0], ReadData2[31:0], WriteEn (tín hiệu cho phép ghi) WriteData[31:0], ReadData1[31:0]. Viết testbench tạo giá trị cho các tín hiệu input và chạy mô phỏng kiểm tra chức năng và timing của thiết kế.

Hiện thực hóa thiết kế:

```
module bai3(
    input wire clk,
    input wire [4:0] ReadAddress1, ReadAddress2,
    input wire WriteEn,
    input wire [4:0] WriteAddress,
    input wire [31:0] WriteData,
    output wire [31:0] ReadData1, ReadData2
);

    reg [31:0] RegFile [31:0];
    always @(posedge clk) begin
        if(WriteEn) begin
            RegFile[WriteAddress] <= WriteData;
        end
    end

    assign ReadData1 = RegFile[ReadAddress1];
```

```
        assign ReadData2 = RegFile[ReadAddress2];  
  
    endmodule
```

Test bench kiểm tra thiết kế có self_checking:

```
`timescale 1ns / 1ps  
  
module tb_bai3;  
  
    // -----  
    // Parameters  
    // -----  
    parameter CLK_PERIOD = 30;  
    parameter HOLD_DELAY = 5; // Giữ dữ liệu thêm 5ns sau cạnh lên để tránh lỗi $hold  
    parameter CHECK_DELAY = 25;  
  
    reg      clk;  
    reg [4:0] ReadAddress1, ReadAddress2;  
    reg      WriteEn;  
    reg [4:0] WriteAddress;  
    reg [31:0] WriteData;  
    wire [31:0] ReadData1, ReadData2;  
  
    integer pass_count;  
    integer fail_count;  
    reg [31:0] ref_mem [0:31];  
  
    bai3 dut (  
        .clk      (clk),  
        .ReadAddress1 (ReadAddress1),  
        .ReadAddress2 (ReadAddress2),  
        .WriteEn     (WriteEn),  
        .WriteAddress (WriteAddress),  
        .WriteData    (WriteData),  
        .ReadData1     (ReadData1),  
        .ReadData2     (ReadData2)  
    );  
  
    // Clock Generation  
    initial clk = 0;  
    always #(CLK_PERIOD/2) clk = ~clk;  
  
    // -----  
    // Task : do_write  
    // -----
```

```

task do_write;
    input [4:0] addr;
    input [31:0] data;
    begin
        @(negedge clk);
        WriteEn    = 1'b1;
        WriteAddress = addr;
        WriteData   = data;

        @(posedge clk);
        #(HOLD_DELAY);
        ref_mem[addr] = data;
        WriteEn      = 1'b0;
    end
endtask

// -----
// Task : do_check
// -----
task do_check;
    input [4:0] addr1;
    input [4:0] addr2;
    input [31:0] exp1;
    input [31:0] exp2;
    input [79:0] label;
    begin
        @(posedge clk);
        #(HOLD_DELAY);
        ReadAddress1 = addr1;
        ReadAddress2 = addr2;

        #(CHECK_DELAY - HOLD_DELAY);

        // Port 1
        if (ReadData1 === exp1) begin
            $display("[PASS] %-10s | Port1[%02d] Got=%08h Exp=%08h", label, addr1,
ReadData1, exp1);
            pass_count = pass_count + 1;
        end else begin
            $display("[FAIL] %-10s | Port1[%02d] Got=%08h Exp=%08h <-- MISMATCH",
label, addr1, ReadData1, exp1);
            fail_count = fail_count + 1;
        end

        // Port 2
        if (ReadData2 === exp2) begin

```

```

        $display("[PASS] %-10s | Port2[%02d] Got=%08h Exp=%08h", label, addr2,
ReadData2, exp2);
        pass_count = pass_count + 1;
    end else begin
        $display("[FAIL] %-10s | Port2[%02d] Got=%08h Exp=%08h <-- MISMATCH",
label, addr2, ReadData2, exp2);
        fail_count = fail_count + 1;
    end
end
endtask

// -----
// Task : do_check1
// -----
task do_check1;
    input [4:0] addr;
    input [31:0] exp;
    input [79:0] label;
    begin
        @(negedge clk);
        ReadAddress1 = addr;
        ReadAddress2 = addr;
        #(CHECK_DELAY);
        if (ReadData1 == exp) begin
            $display("[PASS] %-10s | Port1[%02d] Got=%08h Exp=%08h",
                label, addr, ReadData1, exp);
            pass_count = pass_count + 1;
        end else begin
            $display("[FAIL] %-10s | Port1[%02d] Got=%08h Exp=%08h <-- MISMATCH",
                label, addr, ReadData1, exp);
            fail_count = fail_count + 1;
        end
    end
end
endtask

// -----
// Main stimulus
// -----
integer idx;
reg [4:0] raddr;
reg [31:0] rdata;

initial begin
    // Init
    pass_count = 0;
    fail_count = 0;

```

```

WriteEn    = 1'b0;
WriteAddress = 5'd0;
WriteData   = 32'd0;
ReadAddress1 = 5'd0;
ReadAddress2 = 5'd0;
for (idx = 0; idx < 32; idx = idx + 1)
    ref_mem[idx] = 32'h0;

$display("=====");
$display(" TB_BAI3 32x32 Register File Self-Checking TB");
$display(" CLK=%0dns CHECK_DELAY=%0dns", CLK_PERIOD,
CHECK_DELAY);
$display("=====");

repeat(4) @(posedge clk);

// -----
// T1: Basic write then read back same address on both ports
// -----
$display("\n-- T1: Basic write/read --");
do_write(5'd0, 32'hA5A5A5A5);
do_check(5'd0, 5'd0, 32'hA5A5A5A5, 32'hA5A5A5A5, "T1_BASIC");

// -----
// T2: Multiple independent registers
// -----
$display("\n-- T2: Multiple registers --");
do_write(5'd1, 32'h11111111);
do_write(5'd2, 32'h22222222);
do_write(5'd3, 32'h33333333);
do_write(5'd31, 32'hFFFFFFFF);
do_check(5'd1, 5'd2, 32'h11111111, 32'h22222222, "T2_MULTI");
do_check(5'd3, 5'd31, 32'h33333333, 32'hFFFFFFFF, "T2_MULTI");

// -----
// T3: Overwrite – latest value must appear
// -----
$display("\n-- T3: Overwrite --");
do_write(5'd1, 32'hDEADBEEF);
do_check1(5'd1, 32'hDEADBEEF, "T3_OVWRT");

// -----
// T4: WriteEn=0 must NOT modify register
// -----
$display("\n-- T4: WriteEn=0 no-write --");
@(negedge clk);

```

```

WriteEn    = 1'b0;
WriteAddress = 5'd2;
WriteData   = 32'hBADBAD00;
@(posedge clk);
@(negedge clk);
WriteEn = 1'b0;
// reg[2] must still hold 0x22222222 (written in T2)
do_check1(5'd2, ref_mem[2], "T4_WEN0");

// -----
// T5: Boundary addresses reg[0] and reg[31]
// -----
$display("\n-- T5: Boundary addresses --");
do_write(5'd0, 32'h00000001);
do_write(5'd31, 32'hFFFFFFFE);
do_check(5'd0, 5'd31, 32'h00000001, 32'hFFFFFFFE, "T5_BOUND");

// -----
// T6: Max/Min data values
// -----
$display("\n-- T6: Max/Min data values --");
do_write(5'd10, 32'hFFFFFFFF);
do_write(5'd11, 32'h00000000);
do_check(5'd10, 5'd11, 32'hFFFFFFFF, 32'h00000000, "T6_MXMN");

// -----
// T7: Dual-port read two different registers simultaneously
// -----
$display("\n-- T7: Dual-port simultaneous read --");
do_write(5'd20, 32'hCAFEBABE);
do_write(5'd21, 32'hBEEFCAFE);
do_check(5'd20, 5'd21, 32'hCAFEBABE, 32'hBEEFCAFE, "T7_DUAL");

// -----
// T8: Consecutive writes to same address – last wins
// -----
$display("\n-- T8: Consecutive writes same address --");
do_write(5'd5, 32'h11110000);
do_write(5'd5, 32'h22220000);
do_write(5'd5, 32'h33330000);
do_check1(5'd5, 32'h33330000, "T8_CONS");

// -----
// T9: Write all 32 registers then verify each one
// -----
$display("\n-- T9: Write all 32 registers --");

```

```

for (idx = 0; idx < 32; idx = idx + 1)
    do_write(idx[4:0], 32'hC0000000 | idx[31:0]);

for (idx = 0; idx < 32; idx = idx + 1)
    do_check1(idx[4:0], ref_mem[idx], "T9_ALL32");

// -----
// T10: Random write/read stress – 15 iterations
// -----
$display("\n-- T10: Random stress (15 iters) --");
for (idx = 0; idx < 15; idx = idx + 1) begin
    raddr = $random % 32;
    rdata = $random;
    do_write(raddr, rdata);
    do_check1(raddr, rdata, "T10_RAND");
end

// -----
// Summary
// -----
$display("\n=====");
$display(" SIMULATION COMPLETE");
$display(" PASS : %0d", pass_count);
$display(" FAIL : %0d", fail_count);
$display(" TOTAL : %0d", pass_count + fail_count);
if (fail_count == 0)
    $display(" RESULT: *** ALL TESTS PASSED ***");
else
    $display(" RESULT: !!! %0d TEST(S) FAILED !!!", fail_count);
$display("=====");

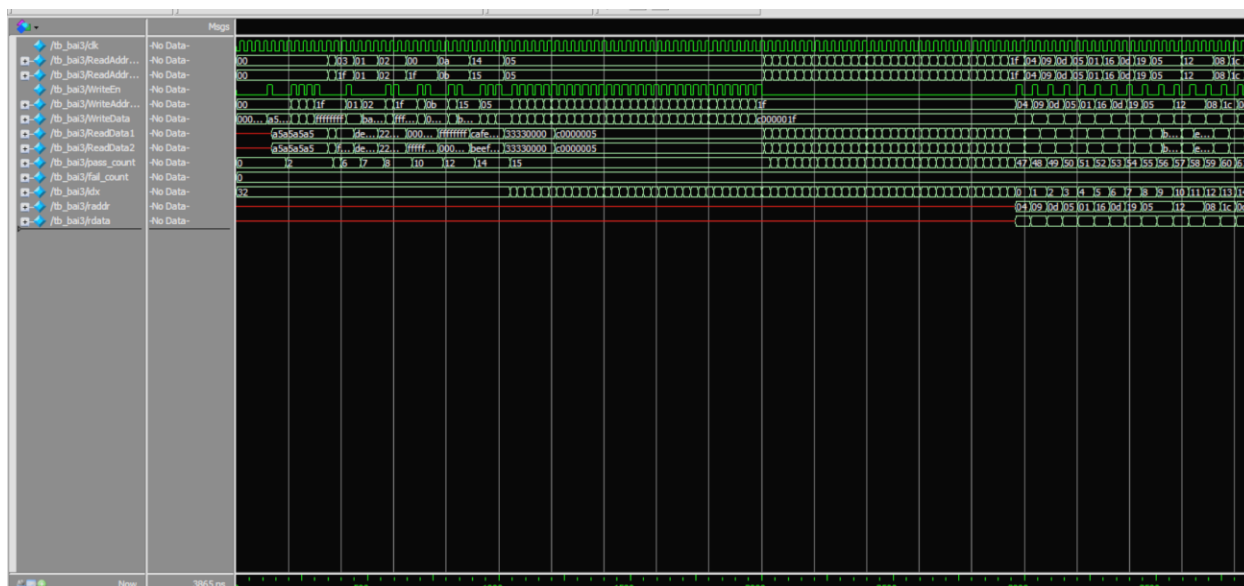
$finish;
end

// -----
// Watchdog – prevent infinite hang
// -----
initial begin
    #500000;
    $display("[WATCHDOG] Simulation timeout!");
    $finish;
end

endmodule

```


Kết quả chạy mô phỏng kiểm tra functional:



Hình 7: Waveform mô phỏng functional

```
# =====
# TB_BAI3 32x32 Register File Self-Checking TB
# CLK=30ns CHECK_DELAY=25ns
# =====
#
# -- T1: Basic write/read --
# [PASS] T1_BASIC | Port1[00] Got=a5a5a5a5 Exp=a5a5a5a5
# [PASS] T1_BASIC | Port2[00] Got=a5a5a5a5 Exp=a5a5a5a5
#
# -- T2: Multiple registers --
# [PASS] T2_MULTI | Port1[01] Got=11111111 Exp=11111111
# [PASS] T2_MULTI | Port2[02] Got=22222222 Exp=22222222
# [PASS] T2_MULTI | Port1[03] Got=33333333 Exp=33333333
# [PASS] T2_MULTI | Port2[31] Got=ffffffff Exp=ffffffff
#
# -- T3: Overwrite --
# [PASS] T3_OVWRT | Port1[01] Got=deadbeef Exp=deadbeef
#
# -- T4: WriteEn=0 no-write --
# [PASS] T4_WEN0 | Port1[02] Got=22222222 Exp=22222222
#
# -- T5: Boundary addresses --
# [PASS] T5_BOUND | Port1[00] Got=00000001 Exp=00000001
# [PASS] T5_BOUND | Port2[31] Got=fffffffe Exp=fffffffe
#
# -- T6: Max/Min data values --
# [PASS] T6_MXMN | Port1[10] Got=ffffffff Exp=ffffffff
```

```
# [PASS] T6_MXMN | Port2[11] Got=00000000 Exp=00000000
#
# -- T7: Dual-port simultaneous read --
# [PASS] T7_DUAL | Port1[20] Got=cafebabe Exp=cafebabe
# [PASS] T7_DUAL | Port2[21] Got=beefcafe Exp=beefcafe
#
# -- T8: Consecutive writes same address --
# [PASS] T8_CONS | Port1[05] Got=33330000 Exp=33330000
#
# -- T9: Write all 32 registers --
# [PASS] T9_ALL32 | Port1[00] Got=c0000000 Exp=c0000000
# [PASS] T9_ALL32 | Port1[01] Got=c0000001 Exp=c0000001
# [PASS] T9_ALL32 | Port1[02] Got=c0000002 Exp=c0000002
# [PASS] T9_ALL32 | Port1[03] Got=c0000003 Exp=c0000003
# [PASS] T9_ALL32 | Port1[04] Got=c0000004 Exp=c0000004
# [PASS] T9_ALL32 | Port1[05] Got=c0000005 Exp=c0000005
# [PASS] T9_ALL32 | Port1[06] Got=c0000006 Exp=c0000006
# [PASS] T9_ALL32 | Port1[07] Got=c0000007 Exp=c0000007
# [PASS] T9_ALL32 | Port1[08] Got=c0000008 Exp=c0000008
# [PASS] T9_ALL32 | Port1[09] Got=c0000009 Exp=c0000009
# [PASS] T9_ALL32 | Port1[10] Got=c000000a Exp=c000000a
# [PASS] T9_ALL32 | Port1[11] Got=c000000b Exp=c000000b
# [PASS] T9_ALL32 | Port1[12] Got=c000000c Exp=c000000c
# [PASS] T9_ALL32 | Port1[13] Got=c000000d Exp=c000000d
# [PASS] T9_ALL32 | Port1[14] Got=c000000e Exp=c000000e
# [PASS] T9_ALL32 | Port1[15] Got=c000000f Exp=c000000f
# [PASS] T9_ALL32 | Port1[16] Got=c0000010 Exp=c0000010
# [PASS] T9_ALL32 | Port1[17] Got=c0000011 Exp=c0000011
# [PASS] T9_ALL32 | Port1[18] Got=c0000012 Exp=c0000012
# [PASS] T9_ALL32 | Port1[19] Got=c0000013 Exp=c0000013
# [PASS] T9_ALL32 | Port1[20] Got=c0000014 Exp=c0000014
# [PASS] T9_ALL32 | Port1[21] Got=c0000015 Exp=c0000015
# [PASS] T9_ALL32 | Port1[22] Got=c0000016 Exp=c0000016
# [PASS] T9_ALL32 | Port1[23] Got=c0000017 Exp=c0000017
# [PASS] T9_ALL32 | Port1[24] Got=c0000018 Exp=c0000018
# [PASS] T9_ALL32 | Port1[25] Got=c0000019 Exp=c0000019
# [PASS] T9_ALL32 | Port1[26] Got=c000001a Exp=c000001a
# [PASS] T9_ALL32 | Port1[27] Got=c000001b Exp=c000001b
# [PASS] T9_ALL32 | Port1[28] Got=c000001c Exp=c000001c
# [PASS] T9_ALL32 | Port1[29] Got=c000001d Exp=c000001d
# [PASS] T9_ALL32 | Port1[30] Got=c000001e Exp=c000001e
# [PASS] T9_ALL32 | Port1[31] Got=c000001f Exp=c000001f
#
# -- T10: Random stress (15 iters) --
# [PASS] T10_RAND | Port1[04] Got=c0895e81 Exp=c0895e81
# [PASS] T10_RAND | Port1[09] Got=b1f05663 Exp=b1f05663
```

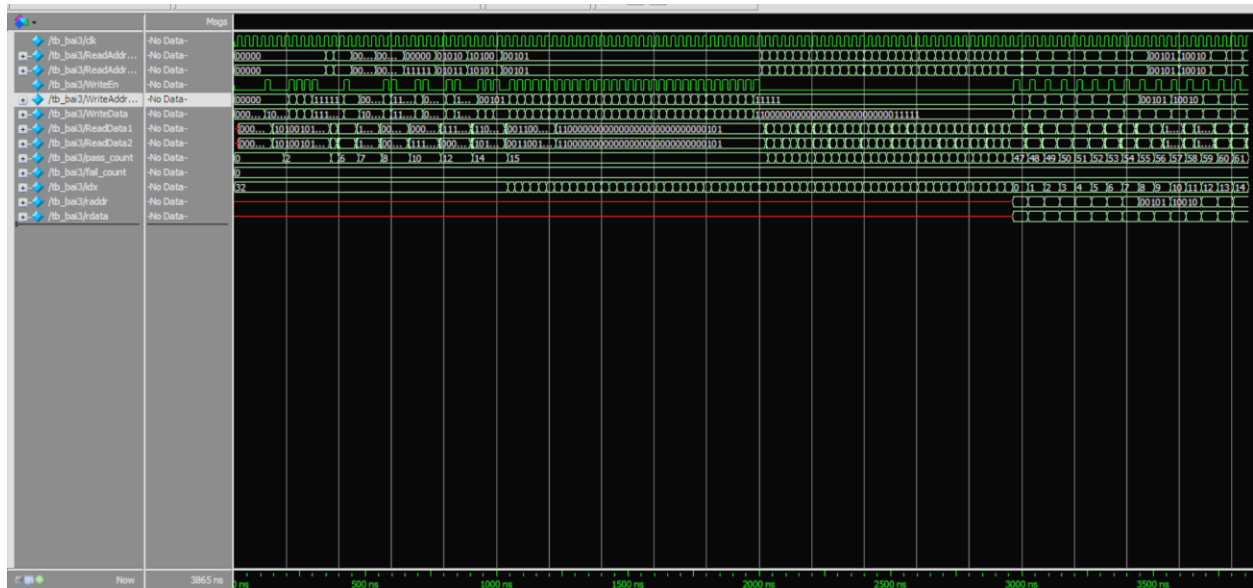
```

# [PASS] T10_RAND | Port1[13] Got=46df998d Exp=46df998d
# [PASS] T10_RAND | Port1[05] Got=89375212 Exp=89375212
# [PASS] T10_RAND | Port1[01] Got=06d7cd0d Exp=06d7cd0d
# [PASS] T10_RAND | Port1[22] Got=1e8dcd3d Exp=1e8dcd3d
# [PASS] T10_RAND | Port1[13] Got=462df78c Exp=462df78c
# [PASS] T10_RAND | Port1[25] Got=e33724c6 Exp=e33724c6
# [PASS] T10_RAND | Port1[05] Got=d513d2aa Exp=d513d2aa
# [PASS] T10_RAND | Port1[05] Got=bbd27277 Exp=bbd27277
# [PASS] T10_RAND | Port1[18] Got=47ecdb8f Exp=47ecdb8f
# [PASS] T10_RAND | Port1[18] Got=e77696ce Exp=e77696ce
# [PASS] T10_RAND | Port1[08] Got=e2ca4ec5 Exp=e2ca4ec5
# [PASS] T10_RAND | Port1[28] Got=de8e28bd Exp=de8e28bd
# [PASS] T10_RAND | Port1[13] Got=b2a72665 Exp=b2a72665
#
# =====
# SIMULATION COMPLETE
# PASS : 62
# FAIL : 0
# TOTAL : 62
# RESULT: *** ALL TESTS PASSED ***
# =====

```

* Nhận xét: Kết quả functional đã chạy đúng

Kết quả chạy mô phỏng timing:



Hình 8: Waveform mô phỏng timing

```

# =====
# TB_BAI3 32x32 Register File Self-Checking TB
# CLK=30ns CHECK_DELAY=25ns
# =====

```

```

#
# -- T1: Basic write/read --
# [PASS] T1_BASIC | Port1[00] Got=a5a5a5a5 Exp=a5a5a5a5
# [PASS] T1_BASIC | Port2[00] Got=a5a5a5a5 Exp=a5a5a5a5
#
# -- T2: Multiple registers --
# [PASS] T2_MULTI | Port1[01] Got=11111111 Exp=11111111
# [PASS] T2_MULTI | Port2[02] Got=22222222 Exp=22222222
# [PASS] T2_MULTI | Port1[03] Got=33333333 Exp=33333333
# [PASS] T2_MULTI | Port2[31] Got=ffffffff Exp=ffffffff
#
# -- T3: Overwrite --
# [PASS] T3_OVWRT | Port1[01] Got=deadbeef Exp=deadbeef
#
# -- T4: WriteEn=0 no-write --
# [PASS] T4_WEN0 | Port1[02] Got=22222222 Exp=22222222
#
# -- T5: Boundary addresses --
# [PASS] T5_BOUND | Port1[00] Got=00000001 Exp=00000001
# [PASS] T5_BOUND | Port2[31] Got=fffffffe Exp=fffffffe
#
# -- T6: Max/Min data values --
# [PASS] T6_MXMN | Port1[10] Got=ffffffff Exp=ffffffff
# [PASS] T6_MXMN | Port2[11] Got=00000000 Exp=00000000
#
# -- T7: Dual-port simultaneous read --
# [PASS] T7_DUAL | Port1[20] Got=cafebabe Exp=cafebabe
# [PASS] T7_DUAL | Port2[21] Got=beefcafe Exp=beefcafe
#
# -- T8: Consecutive writes same address --
# [PASS] T8_CONS | Port1[05] Got=33330000 Exp=33330000
#
# -- T9: Write all 32 registers --
# [PASS] T9_ALL32 | Port1[00] Got=c0000000 Exp=c0000000
# [PASS] T9_ALL32 | Port1[01] Got=c0000001 Exp=c0000001
# [PASS] T9_ALL32 | Port1[02] Got=c0000002 Exp=c0000002
# [PASS] T9_ALL32 | Port1[03] Got=c0000003 Exp=c0000003
# [PASS] T9_ALL32 | Port1[04] Got=c0000004 Exp=c0000004
# [PASS] T9_ALL32 | Port1[05] Got=c0000005 Exp=c0000005
# [PASS] T9_ALL32 | Port1[06] Got=c0000006 Exp=c0000006
# [PASS] T9_ALL32 | Port1[07] Got=c0000007 Exp=c0000007
# [PASS] T9_ALL32 | Port1[08] Got=c0000008 Exp=c0000008
# [PASS] T9_ALL32 | Port1[09] Got=c0000009 Exp=c0000009
# [PASS] T9_ALL32 | Port1[10] Got=c000000a Exp=c000000a
# [PASS] T9_ALL32 | Port1[11] Got=c000000b Exp=c000000b
# [PASS] T9_ALL32 | Port1[12] Got=c000000c Exp=c000000c

```

```
# [PASS] T9_ALL32 | Port1[13] Got=c000000d Exp=c000000d
# [PASS] T9_ALL32 | Port1[14] Got=c000000e Exp=c000000e
# [PASS] T9_ALL32 | Port1[15] Got=c000000f Exp=c000000f
# [PASS] T9_ALL32 | Port1[16] Got=c0000010 Exp=c0000010
# [PASS] T9_ALL32 | Port1[17] Got=c0000011 Exp=c0000011
# [PASS] T9_ALL32 | Port1[18] Got=c0000012 Exp=c0000012
# [PASS] T9_ALL32 | Port1[19] Got=c0000013 Exp=c0000013
# [PASS] T9_ALL32 | Port1[20] Got=c0000014 Exp=c0000014
# [PASS] T9_ALL32 | Port1[21] Got=c0000015 Exp=c0000015
# [PASS] T9_ALL32 | Port1[22] Got=c0000016 Exp=c0000016
# [PASS] T9_ALL32 | Port1[23] Got=c0000017 Exp=c0000017
# [PASS] T9_ALL32 | Port1[24] Got=c0000018 Exp=c0000018
# [PASS] T9_ALL32 | Port1[25] Got=c0000019 Exp=c0000019
# [PASS] T9_ALL32 | Port1[26] Got=c000001a Exp=c000001a
# [PASS] T9_ALL32 | Port1[27] Got=c000001b Exp=c000001b
# [PASS] T9_ALL32 | Port1[28] Got=c000001c Exp=c000001c
# [PASS] T9_ALL32 | Port1[29] Got=c000001d Exp=c000001d
# [PASS] T9_ALL32 | Port1[30] Got=c000001e Exp=c000001e
# [PASS] T9_ALL32 | Port1[31] Got=c000001f Exp=c000001f
#
# -- T10: Random stress (15 iters) --
# [PASS] T10_RAND | Port1[04] Got=c0895e81 Exp=c0895e81
# [PASS] T10_RAND | Port1[09] Got=b1f05663 Exp=b1f05663
# [PASS] T10_RAND | Port1[13] Got=46df998d Exp=46df998d
# [PASS] T10_RAND | Port1[05] Got=89375212 Exp=89375212
# [PASS] T10_RAND | Port1[01] Got=06d7cd0d Exp=06d7cd0d
# [PASS] T10_RAND | Port1[22] Got=1e8dcd3d Exp=1e8dcd3d
# [PASS] T10_RAND | Port1[13] Got=462df78c Exp=462df78c
# [PASS] T10_RAND | Port1[25] Got=e33724c6 Exp=e33724c6
# [PASS] T10_RAND | Port1[05] Got=d513d2aa Exp=d513d2aa
# [PASS] T10_RAND | Port1[05] Got=bbd27277 Exp=bbd27277
# [PASS] T10_RAND | Port1[18] Got=47ecdb8f Exp=47ecdb8f
# [PASS] T10_RAND | Port1[18] Got=e77696ce Exp=e77696ce
# [PASS] T10_RAND | Port1[08] Got=e2ca4ec5 Exp=e2ca4ec5
# [PASS] T10_RAND | Port1[28] Got=de8e28bd Exp=de8e28bd
# [PASS] T10_RAND | Port1[13] Got=b2a72665 Exp=b2a72665
#
# =====
# SIMULATION COMPLETE
# PASS : 62
# FAIL : 0
# TOTAL : 62
# RESULT: *** ALL TESTS PASSED ***
# =====
```

* Nhận xét: Kết quả chạy timing đã chính xác, các test đã được pass hết, từ waveform có thể thấy tại thời điểm khi ReadAddress thay đổi thì ReadData không thay đổi ngay mà có các delay và nhiễu

Câu 4: Sử dụng ngôn ngữ Verilog HDL, hiện thức một dual_port RAM kích thước 1024x8 (dung lượng 1KB) có các tín hiệu sau: Address[9:0], WriteData[7:0], ReadData[7:0], WriteEn, ReadEn. Viết testbench tạo giá trị cho các tín hiệu input và chạy mô phỏng kiểm tra chức năng và timing của thiết kế.

Hiện thực hóa thiết kế:

```
module bai4(
    input wire clk,
    // Cổng Ghi
    input wire [9:0] WriteAddress,
    input wire [7:0] WriteData,
    input wire WriteEn,
    // Cổng Đọc
    input wire [9:0] ReadAddress,
    output reg [7:0] ReadData,
    input wire ReadEn
);
    reg [7:0] ram [0:1023];

    always @(posedge clk) begin
        if (WriteEn) begin
            ram[WriteAddress] <= WriteData;
        end
        if (ReadEn) begin
            ReadData <= ram[ReadAddress];
        end
    end
endmodule
```

Test bench kiểm tra thiết kế có self_checking:

```
`timescale 1ns / 1ps

module tb_bai4;

    // -----
    // Parameters for RTL and Gate-Level Simulation
    // -----
    parameter CLK_PERIOD = 40;
    parameter HOLD_DELAY = 5;
```

```
parameter CHECK_DELAY = 35;
```

```
// -----
```

```
// DUT Signals
```

```
// -----
```

```
reg      clk;
```

```
// Write Port
```

```
reg [9:0] WriteAddress;
```

```
reg [7:0] WriteData;
```

```
reg      WriteEn;
```

```
// Read Port
```

```
reg [9:0] ReadAddress;
```

```
reg      ReadEn;
```

```
wire [7:0] ReadData;
```

```
// -----
```

```
// Reference Model / Scoreboard
```

```
// -----
```

```
reg [7:0] ref_mem [0:1023];
```

```
integer pass_count;
```

```
integer fail_count;
```

```
integer idx;
```

```
// -----
```

```
// DUT Instantiation
```

```
// -----
```

```
bai4 dut (
```

```
    .clk      (clk),
```

```
    .WriteAddress (WriteAddress),
```

```
    .WriteData   (WriteData),
```

```
    .WriteEn     (WriteEn),
```

```
    .ReadAddress (ReadAddress),
```

```
    .ReadData    (ReadData),
```

```
    .ReadEn      (ReadEn)
```

```
);
```

```
// -----
```

```
// Clock Generation
```

```
// -----
```

```
initial clk = 1'b0;
```

```

always #(CLK_PERIOD/2)
    clk = ~clk;

// -----
// TASK : WRITE
// -----
task do_write;
    input [9:0] addr;
    input [7:0] data;

    begin
        @(negedge clk);

        WriteAddress = addr;
        WriteData    = data;
        WriteEn      = 1'b1;

        @(posedge clk);

        #(HOLD_DELAY);

        ref_mem[addr] = data;

        WriteEn = 1'b0;
    end
endtask

// -----
// TASK : READ + SELF CHECK
// -----
task do_read_check;
    input [9:0] addr;
    input [7:0] expected;
    input [79:0] label;

    begin
        @(negedge clk);

        ReadAddress = addr;
        ReadEn      = 1'b1;

        @(posedge clk);

        #(CHECK_DELAY);

        if (ReadData === expected) begin

```



```

        $display("[PASS] %-10s | Addr=%04d | Expected=%02h | Actual=%02h",
            label, addr, expected, ReadData);
        pass_count = pass_count + 1;
    end
    else begin
        $display("[FAIL] %-10s | Addr=%04d | Expected=%02h | Actual=%02h",
            label, addr, expected, ReadData);
        fail_count = fail_count + 1;
    end

    ReadEn = 1'b0;
end
endtask

// -----
// TASK : NO WRITE CHECK
// -----
task do_no_write_check;
    input [9:0] addr;
    input [7:0] expected;
    input [79:0] label;

    begin
        @(negedge clk);

        WriteEn    = 1'b0;
        WriteAddress = addr;
        WriteData   = 8'hFF;

        @(posedge clk);
        #(HOLD_DELAY);

        do_read_check(addr, expected, label);
    end
endtask

// -----
// Monitor
// -----
initial begin
    $monitor("Time=%0t | WE=%b WA=%0d WD=%02h | RE=%b RA=%0d RD=%02h",
        $time,
        WriteEn, WriteAddress, WriteData,
        ReadEn, ReadAddress, ReadData);
end

```

```

// -----
// Main Test Sequence
// -----
reg [9:0] rand_addr;
reg [7:0] rand_data;

initial begin

    // -----
    // Initialization
    // -----
    pass_count = 0;
    fail_count = 0;

    WriteAddress = 10'd0;
    WriteData    = 8'd0;
    WriteEn      = 1'b0;

    ReadAddress  = 10'd0;
    ReadEn       = 1'b0;

    for (idx = 0; idx < 1024; idx = idx + 1)
        ref_mem[idx] = 8'h00;

    $display("=====");
    $display("    TB_BAI4 - SIMPLE DUAL PORT RAM TESTBENCH");
    $display("=====");

    repeat(4) @(posedge clk);

    // -----
    // T1 : Basic Write / Read
    // -----
    $display("\n--- T1 : BASIC WRITE / READ ---");

    do_write(10'd0, 8'hAA);
    do_read_check(10'd0, 8'hAA, "T1_BASIC");

    // -----
    // T2 : Multiple Address Test
    // -----
    $display("\n--- T2 : MULTIPLE ADDRESS TEST ---");

    do_write(10'd1, 8'h11);
    do_write(10'd2, 8'h22);
    do_write(10'd3, 8'h33);

```

```

do_read_check(10'd1, 8'h11, "T2_MULTI");
do_read_check(10'd2, 8'h22, "T2_MULTI");
do_read_check(10'd3, 8'h33, "T2_MULTI");

// -----
// T3 : Overwrite Test
// -----
$display("\n--- T3 : OVERWRITE TEST ---");

do_write(10'd5, 8'h55);
do_write(10'd5, 8'hAA);

do_read_check(10'd5, 8'hAA, "T3_OVR");

// -----
// T4 : Write Enable Disable Test
// -----
$display("\n--- T4 : WRITE ENABLE TEST ---");

do_write(10'd10, 8'h5A);

do_no_write_check(10'd10, 8'h5A, "T4_WEN0");

// -----
// T5 : Boundary Address Test
// -----
$display("\n--- T5 : BOUNDARY TEST ---");

do_write(10'd0, 8'h12);
do_write(10'd1023, 8'hFE);

do_read_check(10'd0, 8'h12, "T5_LOW");
do_read_check(10'd1023, 8'hFE, "T5_HIGH");

// -----
// T6 : Random Stress Test
// -----
$display("\n--- T6 : RANDOM STRESS TEST ---");

for (idx = 0; idx < 20; idx = idx + 1) begin

    rand_addr = $random % 1024;
    rand_data = $random;

    do_write(rand_addr, rand_data);

```

```

        do_read_check(rand_addr, rand_data, "T6_RAND");
    end

    // -----
    // T7 : Sequential Full Sweep
    // -----
    $display("\n--- T7 : FULL SWEEP TEST ---");

    for (idx = 0; idx < 32; idx = idx + 1)
        do_write(idx[9:0], idx[7:0]);

    for (idx = 0; idx < 32; idx = idx + 1)
        do_read_check(idx[9:0], ref_mem[idx], "T7_SWEEP");

    // -----
    // Summary
    // -----
    $display("\n=====");
    $display("SIMULATION COMPLETE");
    $display("PASS = %0d", pass_count);
    $display("FAIL = %0d", fail_count);
    $display("TOTAL = %0d", pass_count + fail_count);

    if (fail_count == 0)
        $display("RESULT : ALL TESTS PASSED");
    else
        $display("RESULT : %0d TEST(S) FAILED", fail_count);

    $display("=====");

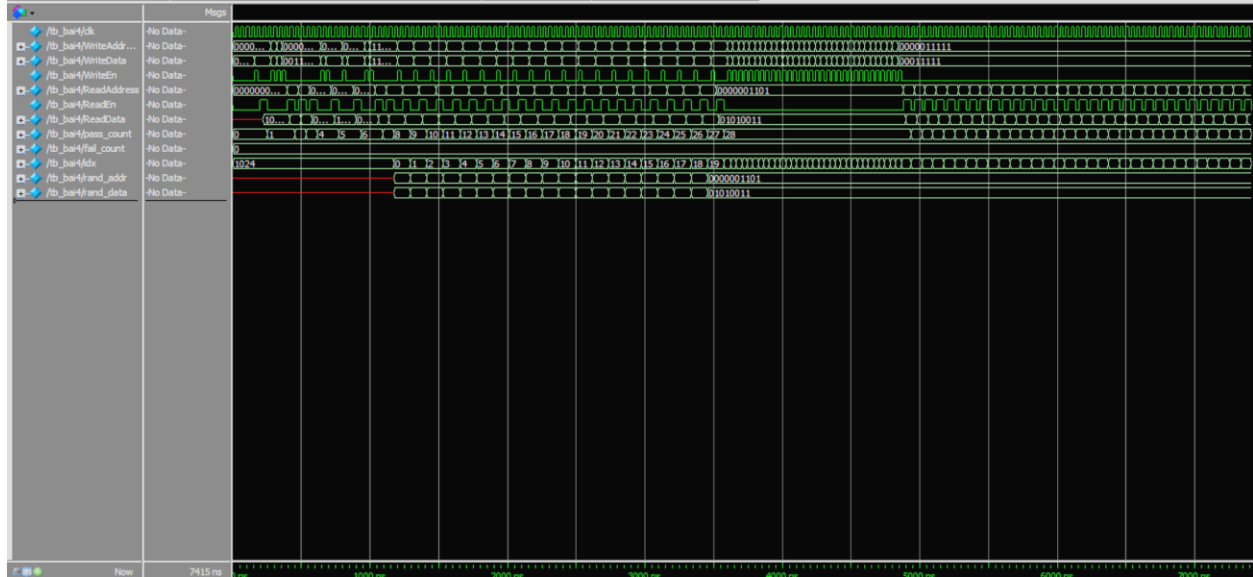
    $finish;
end

// -----
// Watchdog Timer
// -----
initial begin
    #1000000;
    $display("[WATCHDOG] Simulation Timeout!");
    $finish;
end

endmodule

```

Kết quả chạy mô phỏng functional:



Hình 9: Waveform mô phỏng functional

```
# =====
#   TB_BAI4 - SIMPLE DUAL PORT RAM TESTBENCH
# =====
#
# --- T1 : BASIC WRITE / READ ---
# [PASS] T1_BASIC   | Addr=0000 | Expected=aa | Actual=aa
#
# --- T2 : MULTIPLE ADDRESS TEST ---
# [PASS] T2_MULTI   | Addr=0001 | Expected=11 | Actual=11
# [PASS] T2_MULTI   | Addr=0002 | Expected=22 | Actual=22
# [PASS] T2_MULTI   | Addr=0003 | Expected=33 | Actual=33
#
# --- T3 : OVERWRITE TEST ---
# [PASS] T3_OVR     | Addr=0005 | Expected=aa | Actual=aa
#
# --- T4 : WRITE ENABLE TEST ---
# [PASS] T4_WEN0    | Addr=0010 | Expected=5a | Actual=5a
#
# --- T5 : BOUNDARY TEST ---
# [PASS] T5_LOW     | Addr=0000 | Expected=12 | Actual=12
# [PASS] T5_HIGH    | Addr=1023 | Expected=fe | Actual=fe
#
# --- T6 : RANDOM STRESS TEST ---
# [PASS] T6_RAND     | Addr=0292 | Expected=81 | Actual=81
# [PASS] T6_RAND     | Addr=0521 | Expected=63 | Actual=63
# [PASS] T6_RAND     | Addr=0781 | Expected=8d | Actual=8d
# [PASS] T6_RAND     | Addr=0101 | Expected=12 | Actual=12
# [PASS] T6_RAND     | Addr=0769 | Expected=0d | Actual=0d
```

# [PASS] T6 RAND	Addr=0374	Expected=3d	Actual=3d
# [PASS] T6 RAND	Addr=1005	Expected=8c	Actual=8c
# [PASS] T6 RAND	Addr=0505	Expected=c6	Actual=c6
# [PASS] T6 RAND	Addr=0197	Expected=aa	Actual=aa
# [PASS] T6 RAND	Addr=0997	Expected=77	Actual=77
# [PASS] T6 RAND	Addr=0530	Expected=8f	Actual=8f
# [PASS] T6 RAND	Addr=0498	Expected=ce	Actual=ce
# [PASS] T6 RAND	Addr=0744	Expected=c5	Actual=c5
# [PASS] T6 RAND	Addr=0348	Expected=bd	Actual=bd
# [PASS] T6 RAND	Addr=0045	Expected=65	Actual=65
# [PASS] T6 RAND	Addr=0611	Expected=0a	Actual=0a
# [PASS] T6 RAND	Addr=0640	Expected=20	Actual=20
# [PASS] T6 RAND	Addr=0426	Expected=9d	Actual=9d
# [PASS] T6 RAND	Addr=0662	Expected=13	Actual=13
# [PASS] T6 RAND	Addr=0013	Expected=53	Actual=53
#			
# --- T7 : FULL SWEEP TEST ---			
# [PASS] T7 SWEEP	Addr=0000	Expected=00	Actual=00
# [PASS] T7 SWEEP	Addr=0001	Expected=01	Actual=01
# [PASS] T7 SWEEP	Addr=0002	Expected=02	Actual=02
# [PASS] T7 SWEEP	Addr=0003	Expected=03	Actual=03
# [PASS] T7 SWEEP	Addr=0004	Expected=04	Actual=04
# [PASS] T7 SWEEP	Addr=0005	Expected=05	Actual=05
# [PASS] T7 SWEEP	Addr=0006	Expected=06	Actual=06
# [PASS] T7 SWEEP	Addr=0007	Expected=07	Actual=07
# [PASS] T7 SWEEP	Addr=0008	Expected=08	Actual=08
# [PASS] T7 SWEEP	Addr=0009	Expected=09	Actual=09
# [PASS] T7 SWEEP	Addr=0010	Expected=0a	Actual=0a
# [PASS] T7 SWEEP	Addr=0011	Expected=0b	Actual=0b
# [PASS] T7 SWEEP	Addr=0012	Expected=0c	Actual=0c
# [PASS] T7 SWEEP	Addr=0013	Expected=0d	Actual=0d
# [PASS] T7 SWEEP	Addr=0014	Expected=0e	Actual=0e
# [PASS] T7 SWEEP	Addr=0015	Expected=0f	Actual=0f
# [PASS] T7 SWEEP	Addr=0016	Expected=10	Actual=10
# [PASS] T7 SWEEP	Addr=0017	Expected=11	Actual=11
# [PASS] T7 SWEEP	Addr=0018	Expected=12	Actual=12
# [PASS] T7 SWEEP	Addr=0019	Expected=13	Actual=13
# [PASS] T7 SWEEP	Addr=0020	Expected=14	Actual=14
# [PASS] T7 SWEEP	Addr=0021	Expected=15	Actual=15
# [PASS] T7 SWEEP	Addr=0022	Expected=16	Actual=16
# [PASS] T7 SWEEP	Addr=0023	Expected=17	Actual=17
# [PASS] T7 SWEEP	Addr=0024	Expected=18	Actual=18
# [PASS] T7 SWEEP	Addr=0025	Expected=19	Actual=19
# [PASS] T7 SWEEP	Addr=0026	Expected=1a	Actual=1a
# [PASS] T7 SWEEP	Addr=0027	Expected=1b	Actual=1b
# [PASS] T7 SWEEP	Addr=0028	Expected=1c	Actual=1c

```
# [PASS] T7_SWEEP | Addr=0029 | Expected=1d | Actual=1d
# [PASS] T7_SWEEP | Addr=0030 | Expected=1e | Actual=1e
# [PASS] T7_SWEEP | Addr=0031 | Expected=1f | Actual=1f
#
# =====
# SIMULATION COMPLETE
# PASS = 60
# FAIL = 0
# TOTAL = 60
# RESULT : ALL TESTS PASSED
# =====
```

* Nhận xét: Kết quả kiểm tra thiết kế đã chạy chính xác chức năng và pass hết các test case

Kết quả chạy mô phỏng timing:



Hình 10: Waveform mô phỏng timing

```
# =====
# TB_BAI4 - SIMPLE DUAL PORT RAM TESTBENCH
# =====
#
# --- T1 : BASIC WRITE / READ ---
# [PASS] T1_BASIC | Addr=0000 | Expected=aa | Actual=aa
#
# --- T2 : MULTIPLE ADDRESS TEST ---
# [PASS] T2_MULTI | Addr=0001 | Expected=11 | Actual=11
# [PASS] T2_MULTI | Addr=0002 | Expected=22 | Actual=22
# [PASS] T2_MULTI | Addr=0003 | Expected=33 | Actual=33
#
# --- T3 : OVERWRITE TEST ---
# [PASS] T3_OVR | Addr=0005 | Expected=aa | Actual=aa
```

```

#
# --- T4 : WRITE ENABLE TEST ---
# [PASS] T4_WEN0 | Addr=0010 | Expected=5a | Actual=5a
#
# --- T5 : BOUNDARY TEST ---
# [PASS] T5_LOW | Addr=0000 | Expected=12 | Actual=12
# [PASS] T5_HIGH | Addr=1023 | Expected=fe | Actual=fe
#
# --- T6 : RANDOM STRESS TEST ---
# [PASS] T6_RAND | Addr=0292 | Expected=81 | Actual=81
# [PASS] T6_RAND | Addr=0521 | Expected=63 | Actual=63
# [PASS] T6_RAND | Addr=0781 | Expected=8d | Actual=8d
# [PASS] T6_RAND | Addr=0101 | Expected=12 | Actual=12
# [PASS] T6_RAND | Addr=0769 | Expected=0d | Actual=0d
# [PASS] T6_RAND | Addr=0374 | Expected=3d | Actual=3d
# [PASS] T6_RAND | Addr=1005 | Expected=8c | Actual=8c
# [PASS] T6_RAND | Addr=0505 | Expected=c6 | Actual=c6
# [PASS] T6_RAND | Addr=0197 | Expected=aa | Actual=aa
# [PASS] T6_RAND | Addr=0997 | Expected=77 | Actual=77
# [PASS] T6_RAND | Addr=0530 | Expected=8f | Actual=8f
# [PASS] T6_RAND | Addr=0498 | Expected=ce | Actual=ce
# [PASS] T6_RAND | Addr=0744 | Expected=c5 | Actual=c5
# [PASS] T6_RAND | Addr=0348 | Expected=bd | Actual=bd
# [PASS] T6_RAND | Addr=0045 | Expected=65 | Actual=65
# [PASS] T6_RAND | Addr=0611 | Expected=0a | Actual=0a
# [PASS] T6_RAND | Addr=0640 | Expected=20 | Actual=20
# [PASS] T6_RAND | Addr=0426 | Expected=9d | Actual=9d
# [PASS] T6_RAND | Addr=0662 | Expected=13 | Actual=13
# [PASS] T6_RAND | Addr=0013 | Expected=53 | Actual=53
#
# --- T7 : FULL SWEEP TEST ---
# [PASS] T7_SWEEP | Addr=0000 | Expected=00 | Actual=00
# [PASS] T7_SWEEP | Addr=0001 | Expected=01 | Actual=01
# [PASS] T7_SWEEP | Addr=0002 | Expected=02 | Actual=02
# [PASS] T7_SWEEP | Addr=0003 | Expected=03 | Actual=03
# [PASS] T7_SWEEP | Addr=0004 | Expected=04 | Actual=04
# [PASS] T7_SWEEP | Addr=0005 | Expected=05 | Actual=05
# [PASS] T7_SWEEP | Addr=0006 | Expected=06 | Actual=06
# [PASS] T7_SWEEP | Addr=0007 | Expected=07 | Actual=07
# [PASS] T7_SWEEP | Addr=0008 | Expected=08 | Actual=08
# [PASS] T7_SWEEP | Addr=0009 | Expected=09 | Actual=09
# [PASS] T7_SWEEP | Addr=0010 | Expected=0a | Actual=0a
# [PASS] T7_SWEEP | Addr=0011 | Expected=0b | Actual=0b
# [PASS] T7_SWEEP | Addr=0012 | Expected=0c | Actual=0c
# [PASS] T7_SWEEP | Addr=0013 | Expected=0d | Actual=0d
# [PASS] T7_SWEEP | Addr=0014 | Expected=0e | Actual=0e

```



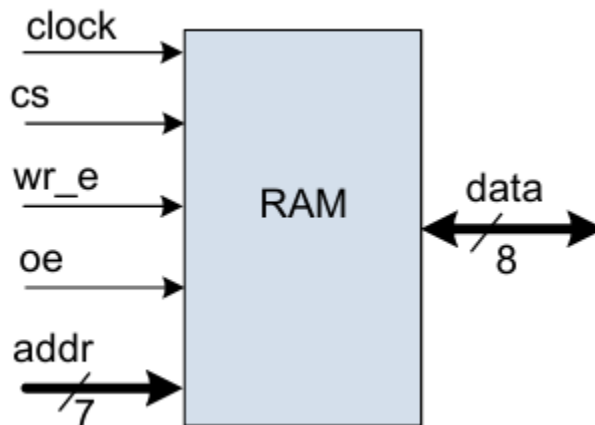
```

# [PASS] T7_SWEEP | Addr=0015 | Expected=0f | Actual=0f
# [PASS] T7_SWEEP | Addr=0016 | Expected=10 | Actual=10
# [PASS] T7_SWEEP | Addr=0017 | Expected=11 | Actual=11
# [PASS] T7_SWEEP | Addr=0018 | Expected=12 | Actual=12
# [PASS] T7_SWEEP | Addr=0019 | Expected=13 | Actual=13
# [PASS] T7_SWEEP | Addr=0020 | Expected=14 | Actual=14
# [PASS] T7_SWEEP | Addr=0021 | Expected=15 | Actual=15
# [PASS] T7_SWEEP | Addr=0022 | Expected=16 | Actual=16
# [PASS] T7_SWEEP | Addr=0023 | Expected=17 | Actual=17
# [PASS] T7_SWEEP | Addr=0024 | Expected=18 | Actual=18
# [PASS] T7_SWEEP | Addr=0025 | Expected=19 | Actual=19
# [PASS] T7_SWEEP | Addr=0026 | Expected=1a | Actual=1a
# [PASS] T7_SWEEP | Addr=0027 | Expected=1b | Actual=1b
# [PASS] T7_SWEEP | Addr=0028 | Expected=1c | Actual=1c
# [PASS] T7_SWEEP | Addr=0029 | Expected=1d | Actual=1d
# [PASS] T7_SWEEP | Addr=0030 | Expected=1e | Actual=1e
# [PASS] T7_SWEEP | Addr=0031 | Expected=1f | Actual=1f
#
# =====
# SIMULATION COMPLETE
# PASS = 60
# FAIL = 0
# TOTAL = 60
# RESULT : ALL TESTS PASSED
# =====

```

* Nhận xét: Kết quả kiểm tra thiết kế đã chạy đúng timing và pass hết các test case

Câu 5: Thiết kế single port RAM đồng bộ read/write



Hình 11 Yêu cầu cụ thể và các chân thiết kế

Hiện thực hóa yêu cầu:

```
module single_port_RAM (
```

```

input clk, cs, wr_e, oe,
input [6:0] addr,
inout [7:0] data
);

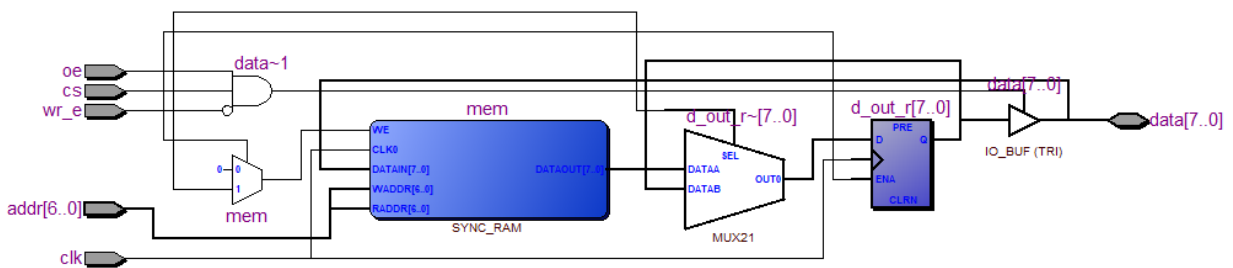
reg [7:0] mem [127:0];
reg [7:0] d_out_r;

always @(posedge clk) begin
    if (cs) begin
        if (wr_e)
            mem[addr] <= data;
        else
            d_out_r <= mem[addr];
    end
end

assign data = (cs && !wr_e && oe) ? d_out_r : 8'bzz;
endmodule

```

Bảng 3 Code verilog thực hiện yêu cầu



Hình 12 RTL Viewer của thiết kế

5.1 Viết testbench để kiểm tra thiết kế theo mô hình quan sát dạng sóng bằng phần mềm ModelSim-Altera

```

`timescale 1ns/1ps

module tb_ram_wave();
    reg clk, cs, wr_e, oe;
    reg [6:0] addr;
    reg [7:0] data_in;
    wire [7:0] data;

    single_port_RAM uut (
        .clk(clk), .cs(cs), .wr_e(wr_e),
        .oe(oe), .addr(addr), .data(data)
    );
endmodule

```

```

);

assign data = (cs && wr_e) ? data_in : 8'bz;

always #5 clk = ~clk;

initial begin
    clk = 0; cs = 0; wr_e = 0; oe = 0; addr = 0; data_in = 0;
    #10;

    $display("TC1: Reading uninitialized memory");
    cs = 1; wr_e = 0; oe = 1;
    addr = 7'h55;
    #10;

    $display("TC2: Write attempt when cs=0");
    cs = 0; wr_e = 1; addr = 7'h01; data_in = 8'hFF;
    #10;
    wr_e = 0; cs = 1; oe = 1; addr = 7'h01;
    #10;

    $display("TC3: Boundary address 127");
    cs = 1; wr_e = 1; addr = 7'd127; data_in = 8'hEE;
    #10;
    wr_e = 0; addr = 7'd127;
    #10;

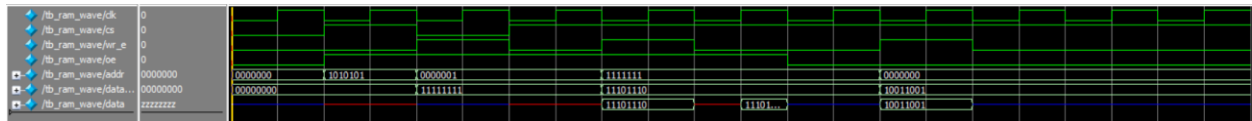
    $display("TC4: Data bus high-Z when oe=0");
    cs = 1; wr_e = 0; oe = 0; addr = 7'd127;
    #10;

    $display("TC5: Address rollover/out of range");
    cs = 1; wr_e = 1; addr = 8'h80; data_in = 8'h99;
    #10;
    addr = 7'h00; wr_e = 0;
    #10;

    #20;
    $display("All corner cases tested.");
    $stop;
end
endmodule

```

Bảng 4 Testbench thực hiện yêu cầu



Hình 13 Waveform cho testbench

- Giải thích waveform:
 - 15ns (cs = 1, oe = 1): Đọc giá trị tại địa chỉ 55 → X vì địa chỉ không lưu dữ liệu
 - 25ns (cs = 0, wr_e = 1, oe = 0): Ghi FF vào địa chỉ 1 → không được ghi do cs ≠ 1
 - 40ns: data = data_in vì là chân inout và được cập nhật bất đồng bộ
 - 55ns (cs = 1, oe = 1): đọc giá trị tại địa chỉ 127
 - 75ns (cs = 1, wr_e = 1): Ghi 99 vào địa chỉ 80, nhưng do cắt bit nên bị ghi đè vào địa chỉ 00
- ⇒ Waveform hoạt động đúng theo yêu cầu → Thiết kế đúng

5.2 Viết testbench để kiểm tra thiết kế theo mô hình tự kiểm tra (Self-checking) bằng phần mềm ModelSim-Altera

```
timescale 1ns/1ps

module tb_ram_self_checking();
    reg clk, cs, wr_e, oe;
    reg [6:0] addr;
    reg [7:0] data_in;
    wire [7:0] data;

    single_port_RAM uut (
        .clk(clk), .cs(cs), .wr_e(wr_e),
        .oe(oe), .addr(addr), .data(data)
    );

    assign data = (cs && wr_e) ? data_in : 8'bz;

    always #5 clk = ~clk;

    task check_result(input [7:0] expected);
        begin
            #1;
            if (data === expected)
                $display("[PASS] Time: %t | Addr: %h | Data: %h", $time, addr, data);
            else
                $display("[FAIL] Time: %t | Addr: %h | Got: %h | Exp: %h", $time, addr, data,
expected);
        end
    endtask
endmodule
```

```

initial begin
    clk = 0; cs = 0; wr_e = 0; oe = 0; addr = 0; data_in = 0;
    #10;

    cs = 1; wr_e = 0; oe = 1; addr = 7'h55;
    #10;
    check_result(8'hxx);

    cs = 0; wr_e = 1; addr = 7'h01; data_in = 8'hFF;
    #10;
    wr_e = 0; cs = 1; oe = 1; addr = 7'h01;
    #10;
    check_result(8'hxx);

    cs = 1; wr_e = 1; addr = 7'd127; data_in = 8'hEE;
    #10;
    wr_e = 0;
    #10;
    check_result(8'hEE);

    cs = 1; wr_e = 0; oe = 0; addr = 7'd127;
    #10;
    check_result(8'hzz);

    wr_e = 1; oe = 1; addr = 7'h00; data_in = 8'h11;
    #10;
    addr = 8'h80; data_in = 8'h99;
    #10;
    wr_e = 0; addr = 7'h00;
    #10;
    check_result(8'h99);

    #20;
    $stop;
end

endmodule

```

Bảng 5 Testbench theo yêu cầu

```

# [PASS] Time:          21000 | Addr: 55 | Data: xx
# [PASS] Time:          42000 | Addr: 01 | Data: xx
# [PASS] Time:          63000 | Addr: 7f | Data: ee
# [PASS] Time:          74000 | Addr: 7f | Data: zz
# [PASS] Time:         105000 | Addr: 00 | Data: 99

```

Hình 14 Console self-checking

Các delay (so với waveform) chủ yếu đến từ task check_result.

Tất cả corners đã pass → Thiết kế đã đúng

Câu 6: Sử dụng chip SRAM trên DE2, kiểm tra quá trình đọc ghi

Chip SRAM trên board DE2 có các đặc điểm:

- Dung lượng: 256K x 16-bit
- Bus địa chỉ: ADDRESS[17:0]
- Bus dữ liệu: DATA[15:0]
- Các tín hiệu điều khiển:
 - o /CE (Chip Enable): Tích cực mức thấp để kích hoạt chip.
 - o /WE (Write Enable): Mức thấp để Ghi, mức cao để Đọc.
 - o /OE (Output Enable): Mức thấp để xuất dữ liệu ra bus DQ.
 - o /UB (Upper Byte) và /LB (Lower Byte): Điều khiển truy cập 8 bit cao hoặc 8 bit thấp.

Thiết kế theo hướng dẫn:

- SW[7:0] làm dữ liệu đầu vào.
- SW[15:8] làm địa chỉ (giải mã ra một phần dung lượng SRAM).
- KEY[0] làm tín hiệu điều khiển Ghi/Đọc.
- HEX0-HEX3 hiển thị giá trị đọc được.

Hiện thực hóa thiết kế:

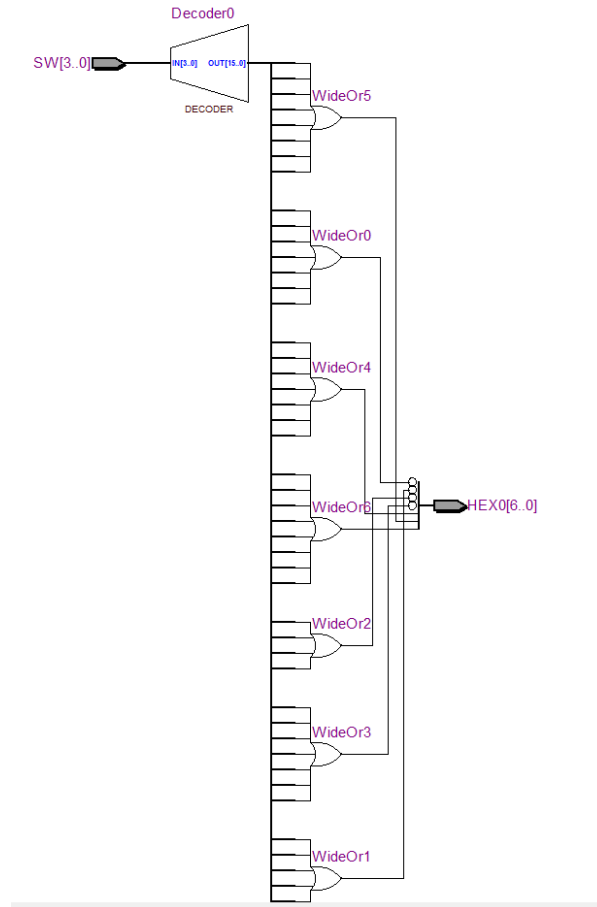
```
module bcd_4bit_to_led7 (  
    input wire [3:0] SW,  
    output reg [6:0] HEX0  
);  
  
always @(*) begin  
    case (SW)  
        4'b0000: HEX0 = 7'b1000000;  
        4'b0001: HEX0 = 7'b11111001;  
        4'b0010: HEX0 = 7'b0100100;  
        4'b0011: HEX0 = 7'b0110000;  
        4'b0100: HEX0 = 7'b0011001;  
        4'b0101: HEX0 = 7'b0010010;  
        4'b0110: HEX0 = 7'b0000010;  
        4'b0111: HEX0 = 7'b11111000;  
        4'b1000: HEX0 = 7'b0000000;  
        4'b1001: HEX0 = 7'b0010000;  
        default: HEX0 = 7'b1111111;  
    endcase  
end
```

```

endcase
end
endmodule

```

Bảng 6 Code verilog giải mã LED 7 đoạn



Hình 15 RTL Viewer thiết kế giải mã LED 7 đoạn

```

module sram_controller (
    input [17:0] SW,      // SW[15:8]: Address, SW[7:0]: Data_in
    input [1:0] KEY,      // KEY[0]: Write Enable (Active Low)
    output [17:0] SRAM_ADDR,
    inout [15:0] SRAM_DQ,
    output SRAM_WE_N, SRAM_OE_N, SRAM_CE_N, SRAM_LB_N, SRAM_UB_N,
    output [6:0] HEX0, HEX1, HEX2, HEX3
);

    assign SRAM_CE_N = 1'b0;
    assign SRAM_LB_N = 1'b0;
    assign SRAM_UB_N = 1'b0;

    assign SRAM_WE_N = KEY[1];
    assign SRAM_OE_N = KEY[0];

```

```

assign SRAM_ADDR = {10'b0, SW[15:8]};

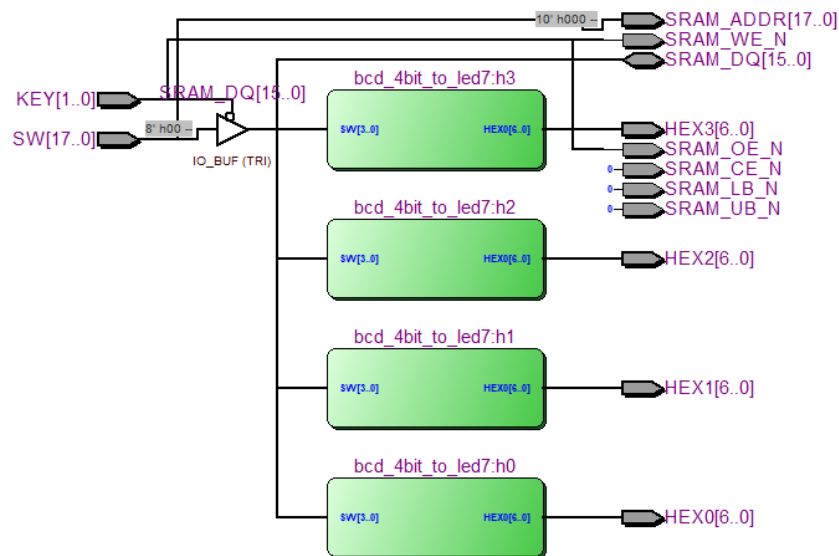
assign SRAM_DQ = (~SRAM_WE_N) ? {8'h00, SW[7:0]} : 16'bz;

bcd_4bit_to_led7 h0 (SRAM_DQ[3:0], HEX0);
bcd_4bit_to_led7 h1 (SRAM_DQ[7:4], HEX1);
bcd_4bit_to_led7 h2 (SRAM_DQ[11:8], HEX2);
bcd_4bit_to_led7 h3 (SRAM_DQ[15:12], HEX3);

endmodule

```

Bảng 7 Code verilog SRAM



Hình 16 RTL Viewer thiết kế SRAM

Thực hiện mô phỏng kiểm tra chức năng:

```

`timescale 1ns/1ps

module tb_sram_controller();
    reg [17:0] SW;
    reg [1:0] KEY;
    wire [17:0] SRAM_ADDR;
    wire [15:0] SRAM_DQ;
    wire SRAM_WE_N, SRAM_OE_N, SRAM_CE_N, SRAM_LB_N, SRAM_UB_N;
    wire [6:0] HEX0, HEX1, HEX2, HEX3;

    sram_controller uut (
        .SW(SW), .KEY(KEY), .SRAM_ADDR(SRAM_ADDR), .SRAM_DQ(SRAM_DQ),

```



```

        .SRAM_WE_N(SRAM_WE_N), .SRAM_OE_N(SRAM_OE_N),
        .SRAM_CE_N(SRAM_CE_N),
        .SRAM_LB_N(SRAM_LB_N), .SRAM_UB_N(SRAM_UB_N),
        .HEX0(HEX0), .HEX1(HEX1), .HEX2(HEX2), .HEX3(HEX3)
    );

    reg [15:0] sram_mem [255:0];
    assign SRAM_DQ = (SRAM_WE_N && !SRAM_OE_N) ?
sram_mem[SRAM_ADDR[7:0]] : 16'bz;

    always @(*) begin
        if (!SRAM_WE_N && !SRAM_CE_N)
            sram_mem[SRAM_ADDR[7:0]] <= SRAM_DQ;
    end

    task check_hex(input [6:0] actual, input [6:0] expected, input [255:0] msg);
        begin
            if (actual === expected)
                $display("[PASS] %s | Got: %b", msg, actual);
            else
                $display("[FAIL] %s | Got: %b | Exp: %b", msg, actual, expected);
        end
    endtask

    initial begin
        SW = 0; KEY = 2'b11;
        #20;

        SW = {8'h05, 8'h09};
        KEY[1] = 0;
        #20;
        KEY[1] = 1;
        #10;

        SW[7:0] = 8'h00;
        KEY[0] = 0;
        #20;
        check_hex(HEX0, 7'b0010000, "Address 05: Check HEX0 (Value 9)");

        KEY[0] = 1;
        #10;
        SW = {8'h05, 8'h03};
        KEY[1] = 0;
        #20;
        KEY[1] = 1;
        #10;
    end

```

```

KEY[0] = 0;
#20;
check_hex(HEX0, 7'b0110000, "Overwrite Address 05: Check HEX0 (Value 3)");

KEY[0] = 1;
#20;
SW = {8'hAA, 8'h07};
KEY = 2'b00;
#20;

if (SRAM_DQ === 16'h0007)
    $display("[PASS] Conflicting Keys (OE=0, WE=0): Write priority observed");
else
    $display("[FAIL] Conflicting Keys: SRAM_DQ = %h", SRAM_DQ);

KEY = 2'b01;
#20;
check_hex(HEX0, 7'b1111000, "Read after Conflict: Check HEX0 (Value 7)");

KEY = 2'b11;
#10;
SW = {8'hFF, 8'h02};
KEY[1] = 0;
#20;
KEY[1] = 1;
#10;

KEY[0] = 0;
#20;
check_hex(HEX0, 7'b0100100, "Address FF: Check HEX0 (Value 2)");

KEY[0] = 1;
#10;
if (SRAM_DQ === 16'hzzzz)
    $display("[PASS] High-Z when OE=1 and WE=1");
else
    $display("[FAIL] Bus not High-Z. Got: %h", SRAM_DQ);

#50;
$display("Simulation Finished.");
$stop;
end

endmodule

```

